Głębokie uczenie ze wzmocnieniem

Piotr Duch

pduch@iis.p.lodz.pl
Instytut Informatyki Stosowanej
Politechnika Łódzka

Lato 2023

Plan wykładu

Informacje ogólne:

- Materiały wykładowe oraz laboratoryjne dostępne są na stronie (pduch.iis.p.lodz.pl).
- Literatura podstawowa:
  - Richard S. Sutton, and Andrew G. Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
  - Morales, Miguel. *Grokking deep reinforcement learning*. Simon and Schuster, 2020.
- Wykłady uzupełniające:
  - RL Course by David Silver - https://www.youtube.com
  - CS 188: Artificial Intelligence by Pieter Abbeel (wykład 10 i 11)- https://www.youtube.com/watch?v=IXuHxkpO5E8
- Materiały dodatkowe:
  - Practical RL Course by Yandex School of Data Analysis - https://github.com/yandexdataschool/Practical_RL
  - CS 188: Introduction to Artificial Intelligence by Berkeley University of California - https://inst.eecs.berkeley.edu/ cs188/fa19/project3/

Informacje ogólne - zaliczenie:

- Laboratorium:
  - Zadania do wykonania w Pythonie (notebooki pythonowe),
  - Ocena końcowa - 66% - 73% ocena 3, 73% - 80% ocena 3.5, 80% - 87% ocena 4, 87% - 94% ocena 4.5, 94% i wyżej - 5.
- Wykład - prezentacja artykułów, odpowiedź ustna.
- Projekt - VizDoom - multi-player mode.
- Kontakt:
  - poprzez platformę MS Teams na chacie indywidualnym,
  - mailowo: pduch@iis.p.lodz.pl.

Informacje szczegółowe - plan działania:

- *Deep Q-Networks* - Notebook Pythonowy 4 (Pliki dodatkowe do notebooków) 29.04.2025.
- *Double Deep Q-Networks* - Notebook Pythonowy 5 29.04.2025.
- *REINFORCE* - Notebook Pythonowy 6 6.05.2025.
- *Actor-Critic* - Notebook Pythonowy 7 6.05.2025.
- *Breakout* - Notebook Pythonowy 8 13.05.2025.

Wprowadzenie

Uczenie ze wzmocnieniem - wprowadzenie

Metody

- Uczenie pasywne:
  - Ocena strategii (ang. *Policy Evaluation*)
  - Polepszanie strategii (ang. *Policy Improvement*)
  - Iteracyjne doskonalenie strategii (ang. *Policy Iteration*)
  - Iteracyjne obliczanie funkcji wartości (ang. *Value Iteration*)

Uczenie ze wzmocnieniem - wprowadzenie

Metody cd.

- Uczenie aktywne:
  - Metody różnic czasowych (ang. *Temporal Difference Learning*)
    - Monte Carlo
    - Q-Learning
    - SARSA
  - Metody aproksymacyjne
    - Aproksymacja funkcji wartości (ang. *Approximate Q-Learning*)

Uczenie ze wzmocnieniem - wprowadzenie

Metody cd.

- Uczenie aktywne:
  - Metody wykorzystujące głębokie sieci neuronowe
    - Deep Q-Learning
    - Double Q-Learning
    - Dueling DQN
    - C51 (Categorical DQN)
    - Quantile Regression DQN
    - Implicit Quantile Networks (IQN)
    - Hindsight Experience Replay (HER)
    - Noisy Networks for Exploration
    - Rainbow DQN
    - Convolutional Deep Q-Learning

Uczenie ze wzmocnieniem - wprowadzenie

Metody cd.

- Uczenie aktywne:
  - Metody wykorzystujące głębokie sieci neuronowe (Policy-Based Methods)
    - REINFORCE (Monte Carlo Policy Gradient)
    - Trust Region Policy Optimization (TRPO)
    - Proximal Policy Optimization (PPO)
    - Actor-Critic
    - A2C (Advantage Actor-Critic)
    - A3C (Asynchronous Advantage Actor-Critic)
    - DDPG (Deep Deterministic Policy Gradient)
    - TD3 (Twin Delayed DDPG)
    - SAC (Soft Actor-Critic)

Uczenie ze wzmocnieniem - wprowadzenie

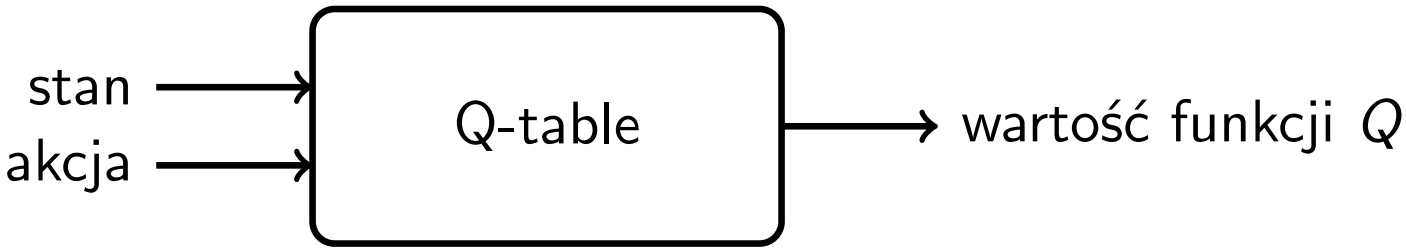
Metody cd.

- Imitation Learning:
  - Behavioral Cloning
  - DAgger
  - AggreVaTe
  - Inverse Reinforcement Learning
  - Generative Adversarial Imitation Learning
  - Adversarial Inverse Reinforcement Learning (AIRL)
- Model-Based RL:
  - Dyna-Q
  - MuZero
  - SimPLe
  - Dreamer V2

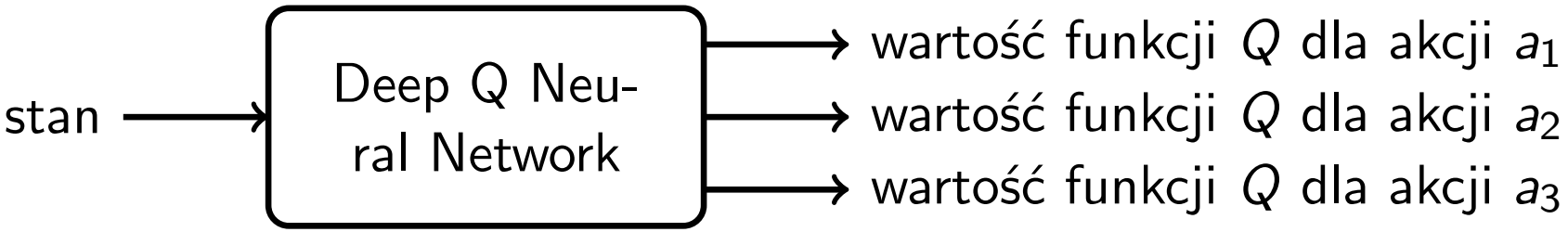
Głębokie uczenie ze wzmocnieniem

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network



Rysunek 1: Q-Learning



Rysunek 2: Deep Q Neural Network

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network

Sposób działania:

- na wejście sieci podawany jest aktualny stan,
- następna akcja jest wybierana na podstawie odpowiedzi sieci (akcja o największej wartości),
- funkcją kosztu jest błąd średniokwadratowy z wartości przewidzianej przez sieć, a tej docelowej (wyznaczonej za pomocą równania Bellmana) - $R + \gamma max_a(Q(s', a))$,
- jako wartość oczekiwaną funkcji *predict* podajemy wektor wartości Q zwrócony przez sieć dla danego stanu, z zaktualizowaną wartością dla wybranej akcji.

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network

Problemy z podejściem DQN:

- sekwencyjnie skorelowane dane - może wpływać na zbieżność i wydajność sieci,
- niestabilność dystrybucji danych z powodu zmian strategii - strategia może nie być zbieżna lub oscylować.

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network - Experience Replay

Rozwiązanie problemu sekwencyjnie skorelowanych danych - zapamiętanie całej serii wykonanych działań, to znaczy stanu (s), akcji podjętej w tym stanie (a), otrzymanej nagrody (r) oraz kolejnego stanu (s'). Uczenie sieci na podstawie losowo dobranych próbek z zebranych danych.

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network - Experience Replay

Sposób działania:

1. zebranie zbioru danych w postaci krotek - (s, a, r, s') na podstawie akcji podjętych przez agenta (sieć), korzystając z algorytmu ϵ-zachłannego,
2. wybór losowych krotek z zebranego zbioru,
3. trenowanie modelu sieci na podstawie wybranych danych,
4. podejmowanie nowych akcji na podstawie zaktualizowanego modelu i algorytmu ϵ-zachłannego,
5. powrót do punktu 2.

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network - Prioritized Experience Replay

Sposób działania:

1. wybierz akcję a w stanie s i zapamiętaj nowy stan (s') oraz otrzymaną nagrodę (r),
2. wyznacz za pomocą sieci wartość dla aktualnego stanu $Q(s, a)$,
3. wyznacz za pomocą sieci oczekiwaną wartość dla aktualnego stanu $target = r + \gamma * max'_a Q(s', a')$,
4. wyznacz wartość absoltuną błędu dla analizowanego stanu $atd\_err = abs(Q(s, a) - target)$,
5. dodaj krotkę (s, a, r, s', atd_err) do bufora,
6. wykorzystaj dane z bufora do uczenia sieci.

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network - Prioritized Experience Replay

Opcje priorytetyzacji:

1. *Proportional prioritization*,
2. *Rank-based prioritization*,
3. *Weighted importance-sampling*.

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network - Prioritized Experience Replay

Proportional prioritization:

$$p_i = |\delta_i| + \epsilon \tag{1}$$

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \tag{2}$$

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network - Prioritized Experience Replay

Rank-based prioritization:

$$\theta = \theta - \alpha \frac{dloss}{d\theta} w \tag{3}$$

$$w(i) = (\frac{1}{N} * \frac{1}{P(i)})^b \tag{4}$$

Głębokie uczenie ze wzmocnieniem

Deep Q Neural Network - Prioritized Experience Replay

Weighted importance-sampling:

$$p(i) = \frac{1}{rank(1)} \tag{5}$$

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \tag{6}$$

Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network

Rozwiązanie problemu niestabilności dystrybucji danych z powodu zmian strategii, ocena wybranej akcji odbywa się za pomocą innej sieci, niż sam wybór akcji.

Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network

Korzystanie z dwóch sieci:

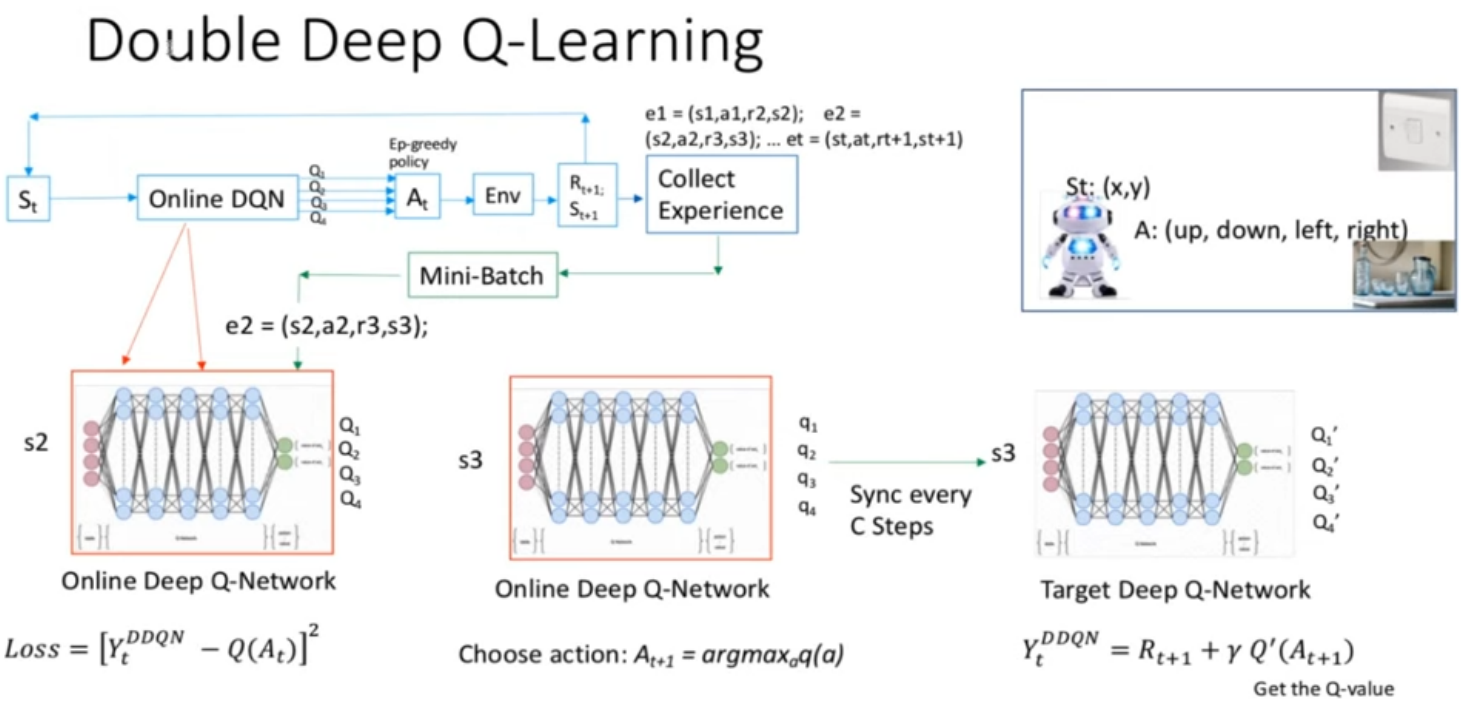
- sieć Q' - wykorzystywana do wyboru akcji podejmowanych przez agenta,
- sieć Q - wykorzystywana do oszacowania wybranej akcji.

Funkcją kosztu jest błąd średniokwadratowy z wartości przewidzianej przez sieć Q', a tej oszacowanej prze sieć Q (wyznaczonej za pomocą równania Bellmana):

$$Q^*(s_t, a_t) \approx r_t + \gamma Q(s_{t+1}, argmax_{a'} Q'(s_{t+1}, a'))$$

Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network



Rysunek 3: Double Deep Q-Learning

[DDQN] Deep Reinforcement Learning with Double Q-learning — TDLS Foundational)

Uczenie aktywne

Double Deep Q Neural Network

Double Q-Learning

Initialize primary network Q_θ, target network Q'_θ, replay buffer D, $\tau << 1$.
For each iteration do:
For each environment step do:
Observe state s and select action a using network Q_θ:
Take action a, observe r, s'
Store (s, a, r, s') in replay buffer D
For each update step do:
Sample $e = (s, a, r, s)$ from D
Compute target Q value:

$$Q^*(s, a) \approx r + \gamma Q(s', argmax_{a'} Q'_\theta(s', a'))$$

Perform gradient descent step on $(Q^*(s, a) - Q_\theta(s, a))^2$
Update target network parameters:

$$\theta' \leftarrow \tau * \theta + (1 - \tau) * \theta'$$

Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. Nature 518, 529–533 (2015).

Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks

$$
\begin{aligned}
Q^{\pi}(s,a) &= \mathbb{E}[R_t \mid s_t = s, a_t = a, \pi] \\
&= \mathbb{E}\left[r + \gamma \mathbb{E}_{a' \sim \pi}\left[Q^{\pi}(s',a') \mid s', a'\right] \mid s_t = s, a_t = a, \pi\right]
\end{aligned} \tag{7}
$$

Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks

$$
\begin{aligned}
Q^{\pi}(s,a) &= \mathbb{E}[R_t \mid s_t = s, a_t = a, \pi] \\
&= \mathbb{E}\left[r + \gamma \mathbb{E}_{a' \sim \pi}\left[Q^{\pi}(s',a') \mid s', a'\right] \mid s_t = s, a_t = a, \pi\right]
\end{aligned} \tag{7}
$$

$$
V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^{\pi}(s,a)] \tag{8}
$$

Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks

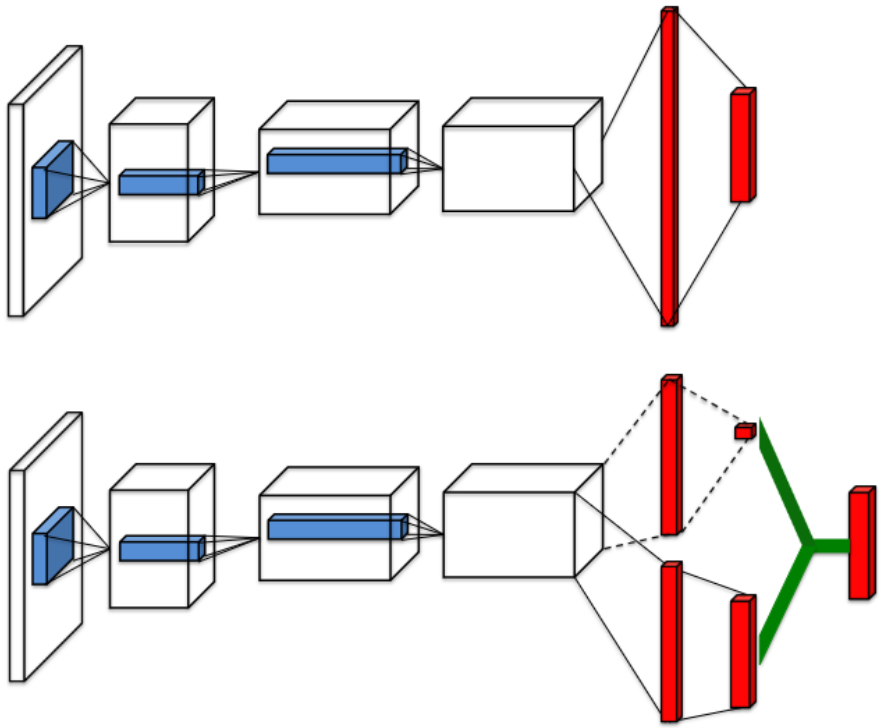
$$\begin{aligned} Q^{\pi}(s,a) &= \mathbb{E}[R_t \mid s_t = s, a_t = a, \pi] \\ &= \mathbb{E}\left[r + \gamma \mathbb{E}_{a' \sim \pi}\left[Q^{\pi}(s',a') \mid s', a'\right] \mid s_t = s, a_t = a, \pi\right] \end{aligned} \tag{7}$$

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^{\pi}(s,a)] \tag{8}$$

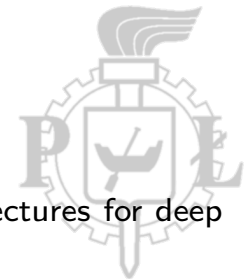
$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s) \tag{9}$$

Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks



Rysunek 4: Dueling Deep Q-Networks



Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016, June). Dueling network architectures for deep reinforcement learning. In International conference on machine learning (pp. 1995-2003). PMLR.

Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s) \tag{10}$$

Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s) \tag{10}$$

$$Q^{\pi}(s,a;\theta,\alpha,\beta) = V^{\pi}(s;\theta,\beta) + A^{\pi}(s,a;\theta,\alpha) \tag{11}$$

Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s) \tag{10}$$

$$Q^{\pi}(s,a;\theta,\alpha,\beta) = V^{\pi}(s;\theta,\beta) + A^{\pi}(s,a;\theta,\alpha) - \max_{a' \in \mathcal{A}} A(s,a';\theta,\alpha) \tag{12}$$

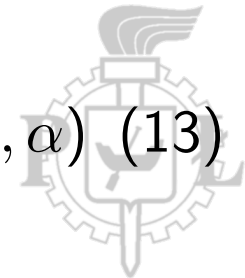
Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s) \tag{10}$$

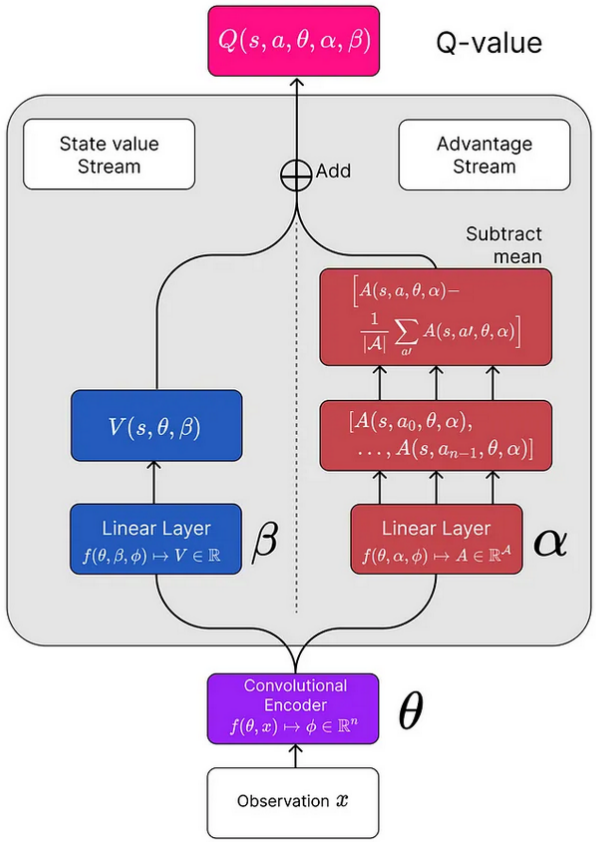
$$Q^{\pi}(s,a;\theta,\alpha,\beta) = V^{\pi}(s;\theta,\beta) + A^{\pi}(s,a;\theta,\alpha) - \max_{a' \in \mathcal{A}} A(s,a';\theta,\alpha) \tag{12}$$

$$Q^{\pi}(s,a;\theta,\alpha,\beta) = V^{\pi}(s;\theta,\beta) + A^{\pi}(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a';\theta,\alpha) \tag{13}$$



Głębokie uczenie ze wzmocnieniem

Dueling Deep Q-Networks



Rysunek 5: Dueling Deep Q-Networks

Ryan Pégoud. Rainbow: The Colorful Evolution of Deep Q-Networks — medium.com. `https://medium.com/towards-data-science/rainbow-the-colorful-evolution-of-deep-q-networks-37e662ab99b2`

Głębokie uczenie ze wzmocnieniem

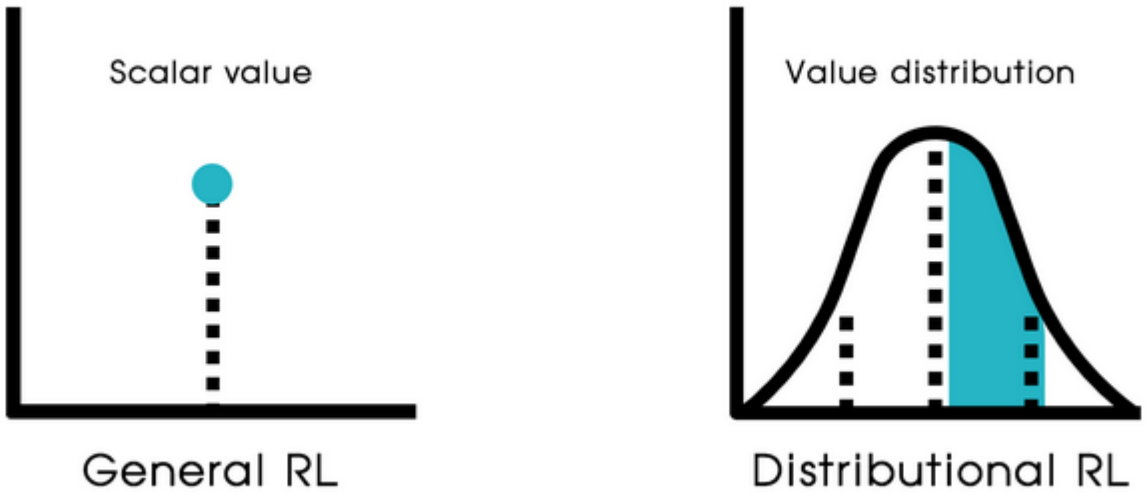
C51 (Categorical DQN)

Problem:

- Action 1: 80% szansy na nagrodę +10, 20% szansy na nagrodę -10.
- Action 2: zawsze +2.

Głębokie uczenie ze wzmocnieniem

C51 (Categorical DQN)



Rysunek 6: C51

Deep Learning Bible - 5. Reinforcement Learning `https://wikidocs.net/book/8154`

Głębokie uczenie ze wzmocnieniem

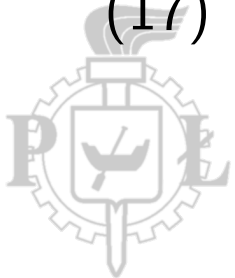
C51 (Categorical DQN)

$$z_i = R_{min} + i\frac{R_{max} - R_{min}}{N-1}, i = 0, 1, 2, ..., N-1 \tag{14}$$

$$Q(s,a) = \sum_{i=0}^{N-1} p_i z_i \tag{15}$$

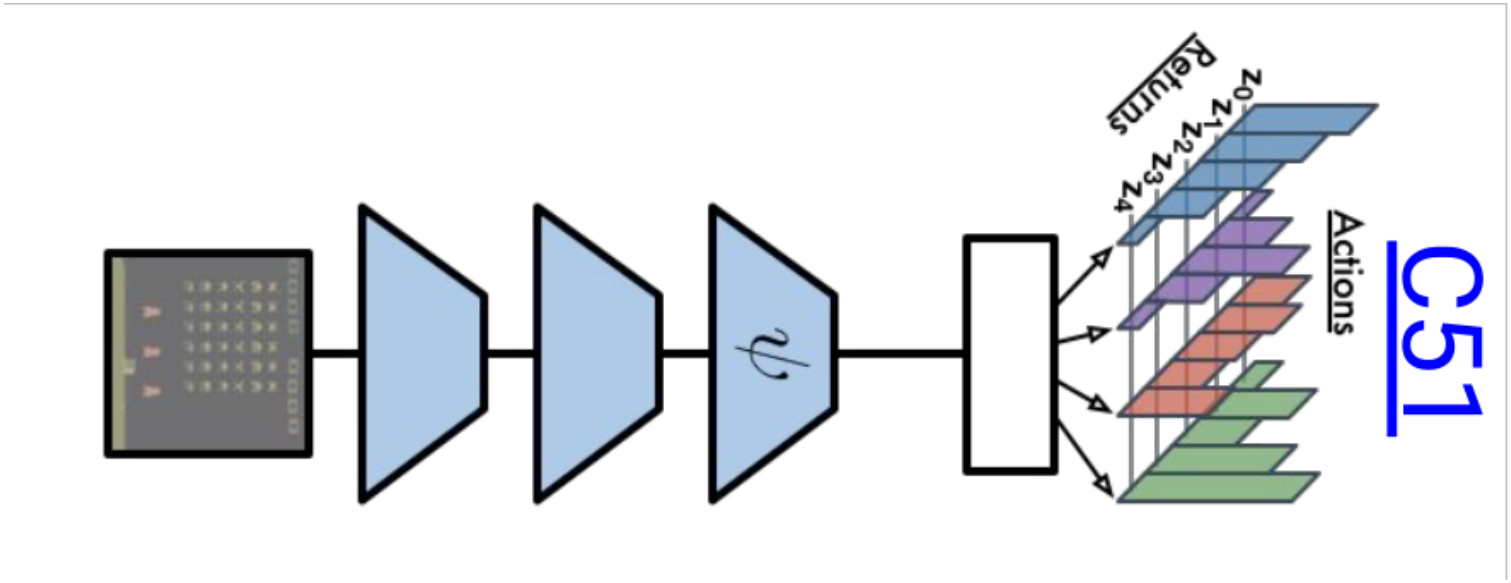
$$Z(s,a) \stackrel{D}{=} R + \gamma Z(s', a^*) \tag{16}$$

$$L(\theta) = -\sum_{i=0}^{50} m_i \log p_\theta(z_i \mid s, a) \tag{17}$$



Głębokie uczenie ze wzmocnieniem

C51 (Categorical DQN)



Rysunek 7: C51

Bellemare, M. G., Dabney, W., & Munos, R. (2017, July). A distributional perspective on reinforcement learning. In International conference on machine learning (pp. 449-458). PMLR.

Głębokie uczenie ze wzmocnieniem

Quantile Regression DQN

- C51 - zakłada, że rozkład nagrody jest reprezentowany przez 51 stałych wartości.
- problem - te wartości mogą ulegać zmianie.

Głębokie uczenie ze wzmocnieniem

Quantile Regression DQN

- C51 - zakłada, że rozkład nagrody jest reprezentowany przez 51 stałych wartości.
- problem - te wartości mogą ulegać zmianie.

Uczmy się kwantyli rozkładu bezpośrednio!

Głębokie uczenie ze wzmocnieniem

Quantile Regression DQN

$$Z_\theta(s,a) = \hat{z}_1, \hat{z}_2, ..., \hat{z}_N \tag{18}$$

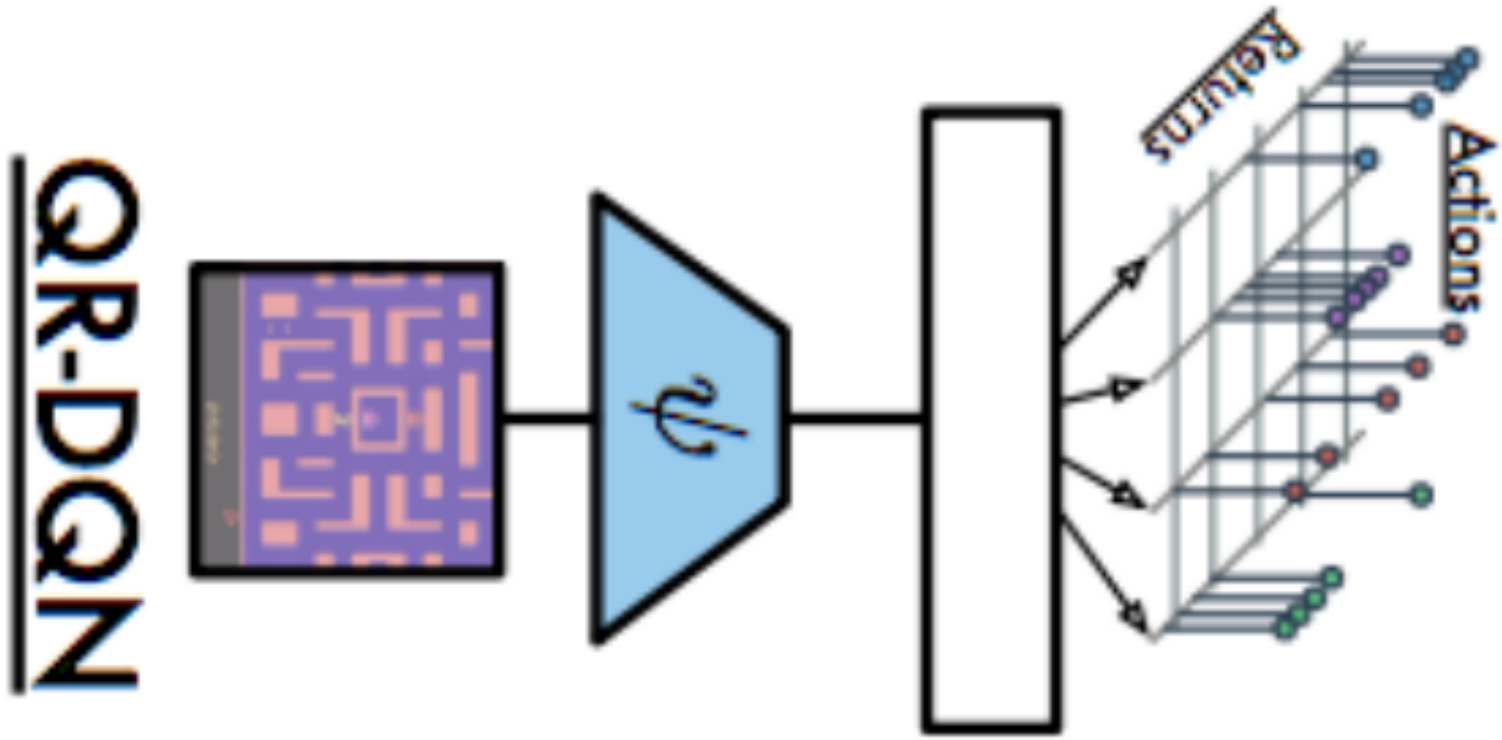
$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{N} \rho_\tau(z_i - \hat{z}_j) \tag{19}$$

Regression loss:

$$\rho_\tau(u) = \begin{cases} \tau u, & u > 0 \\ (\tau - 1)u, & u \leqslant 0 \end{cases} \tag{20}$$

Głębokie uczenie ze wzmocnieniem

Quantile Regression DQN



Rysunek 8: Quantile Regression DQN

Dabney, W., Rowland, M., Bellemare, M., & Munos, R. (2018, April). Distributional reinforcement learning with quantile regression. In Proceedings of the AAAI conference on artificial intelligence (Vol. 32, No. 1).

Głębokie uczenie ze wzmocnieniem

Rainbow DQN

"Jeden, by wszystkimi rządzić, Jeden, by wszystkie odnaleźć, Jeden, by wszystkie zgromadzić i w ciemności związać"

Głębokie uczenie ze wzmocnieniem

Rainbow DQN

- Double DQN - Reduces overestimation bias.
- Dueling DQN - Helps learn state values better.
- Prioritized Replay - Focuses on important experiences.
- Multi-step Learning - Looks ahead multiple steps.
- Distributional RL - Learns full reward distributions.
- Noisy Networks - Replaces ϵ-greedy with better exploration.

Głębokie uczenie ze wzmocnieniem

Rainbow DQN

1. Play & Store Experience - Collects (s, a, r, s')
2. Sample from Replay Buffer - Uses Prioritized Experience Replay
3. Compute Target Q-Values - Target Network Q_θ - Stabilizes Q-value estimates (Dueling Q-Network)
4. Predict Q-Values - Online Network Q_θ - Uses Distributional RL (Dueling Q-Network)
5. Compute Loss - Online Network Q_θ - Uses Quantile Regression Loss
6. Update Online Network - Online Network Q_θ - Backpropagation updates weights
7. Update Target Network - Target Network Q_θ - Slowly copied from Online Network

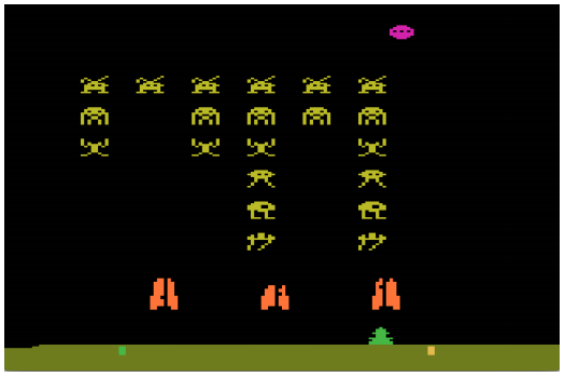
Głębokie uczenie ze wzmocnieniem

Materiały uzupełniające

- Książka *Grokking Deep Reinforcement Learning*, Miguel Morales, October 2020.
  - Rozdziały 9 - *More stable value-based methods* - DQN, DDQN, Experience Replay.
  - Rozdziały 10 - *Sample-efficient value-based methods* - Prioritized Experience Replay.

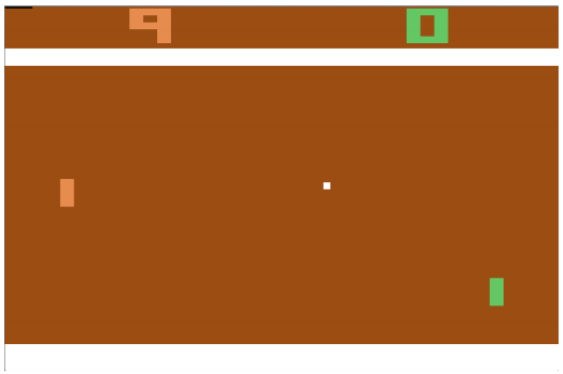
Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network



(a) Space Invaders

(b) Beam Rider



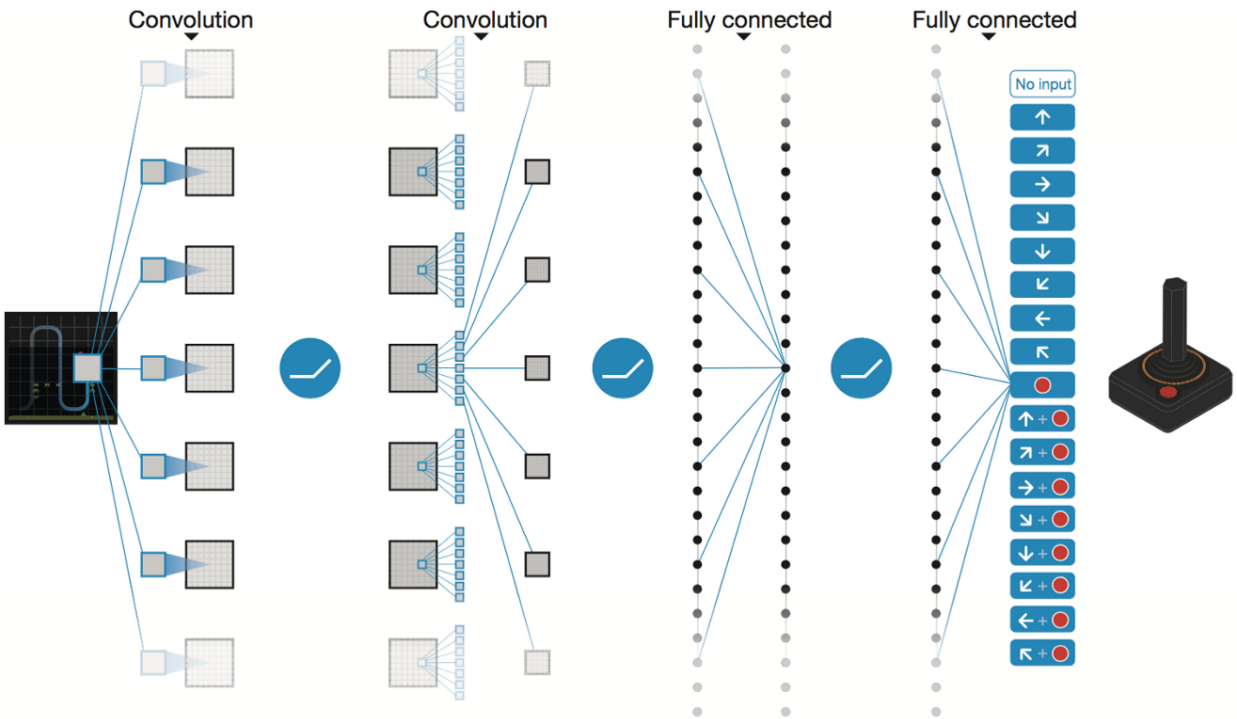
(c) Pong

(d) Breakout

Rysunek 9: Mnih, Volodymyr, et al. "Playing atari with deep reinforcement learning." (2013).

Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network

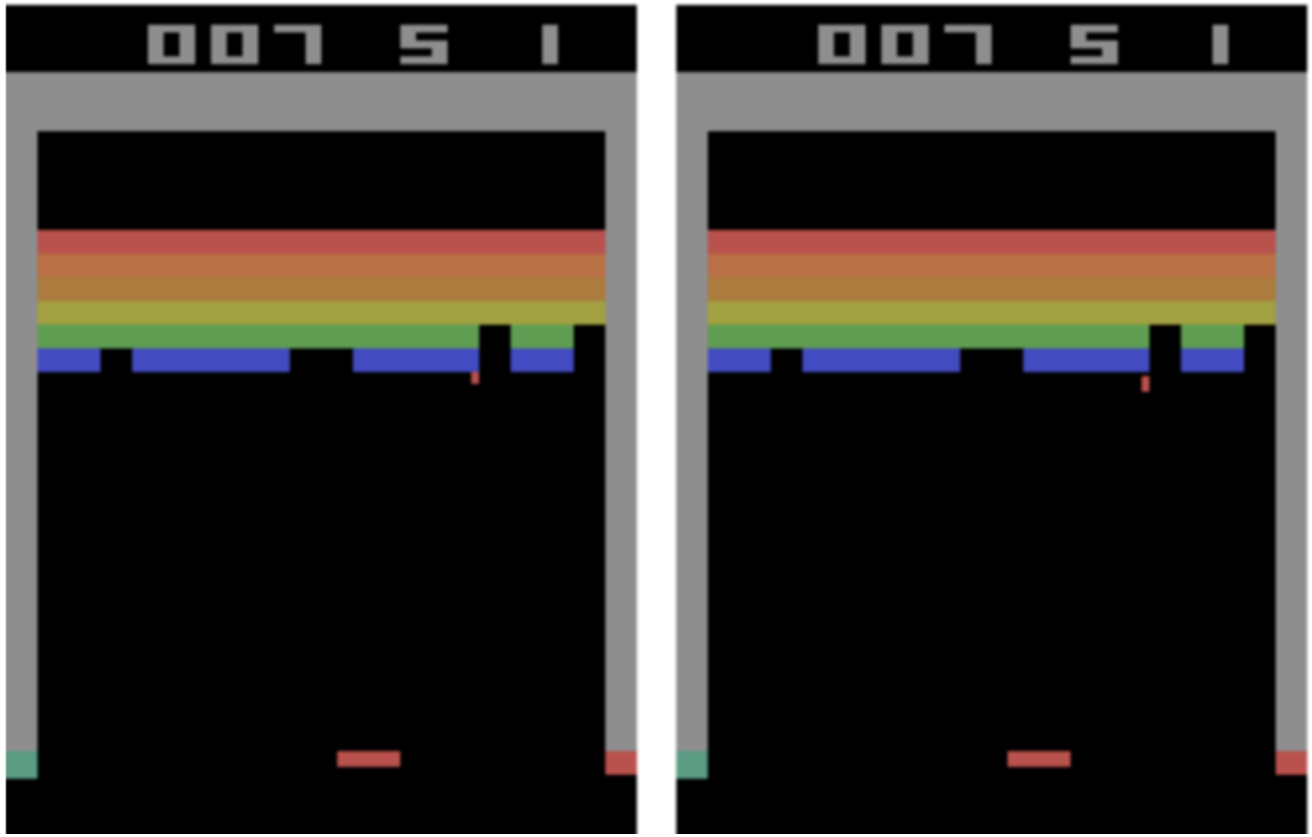


Rysunek 10: Network Architecture

Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. Nature 518, 529–533 (2015).

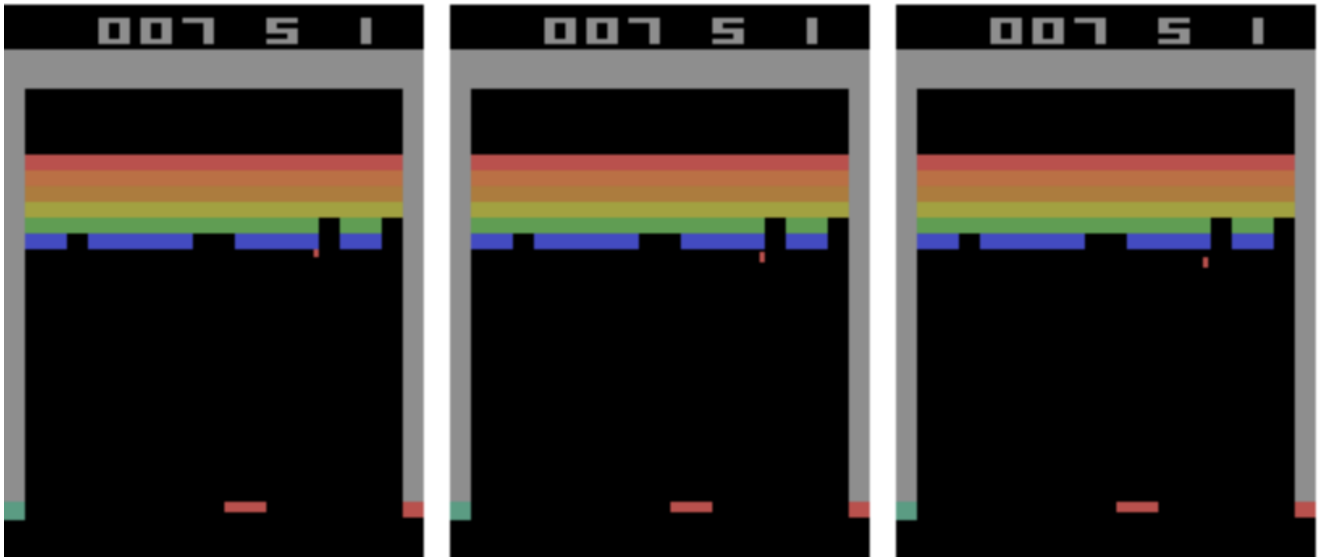
Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network



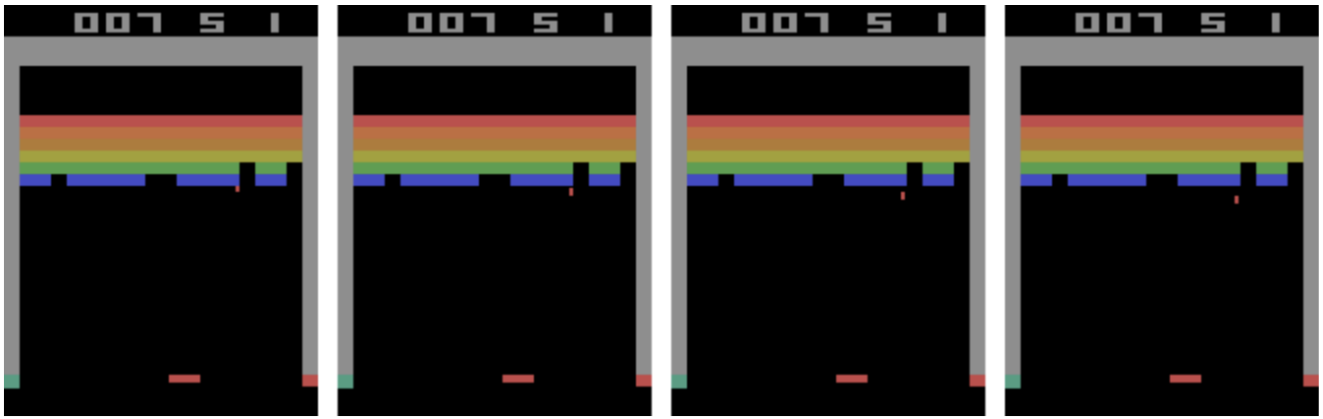
Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network



Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network



Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network

Przetwarzania wstępne obrazów:

- wycięcie istotnego fragmentu obrazu,
- zamiana na odcienie szarości,
- zmniejszenie obrazu.

Głębokie uczenie ze wzmocnieniem

Double Deep Q Neural Network

Hyperparameter	Value	Description
minibatch size	32	Number of training cases over which each stochastic gradient descent (SGD) update is computed.
replay memory size	1000000	SGD updates are sampled from this number of most recent frames.
agent history length	4	The number of most recent frames experienced by the agent that are given as input to the Q network.
discount factor	0.99	Discount factor gamma used in the Q-learning update.
action repeat	4	Repeat each action selected by the agent this many times. Using a value of 4 results in the agent seeing only every 4th input frame.
update frequency	4	The number of actions selected by the agent between successive SGD updates. Using a value of 4 results in the agent selecting 4 actions between each pair of successive updates.
learning rate	0.00025	The learning rate used by RMSProp.
gradient momentum	0.95	Gradient momentum used by RMSProp.
squared gradient momentum	0.95	Squared gradient (denominator) momentum used by RMSProp.
min squared gradient	0.01	Constant added to the squared gradient in the denominator of the RMSProp update.
initial exploration	1	Initial value of ε in ε-greedy exploration.
final exploration	0.1	Final value of ε in ε-greedy exploration.
final exploration frame	1000000	The number of frames over which the initial value of ε is linearly annealed to its final value.
replay start size	50000	A uniform random policy is run for this number of frames before learning starts and the resulting experience is used to populate the replay memory.
no-op max	30	Maximum number of "do nothing" actions to be performed by the agent at the start of an episode.

Rysunek 11: Przykładowe parametry sieci

Mnih, V., Kavukcuoglu, K., Silver, D. et al. Human-level control through deep reinforcement learning. Nature 518, 529–533 (2015).

Głębokie uczenie ze wzmocnieniem

Policy Gradients

Dotychczasowe podejście:

- Aproksymacja funkcji wartości:

$$v_*(s) \approx v_\pi(s)$$

$$q_*(s,a) \approx q_\pi(s,a)$$

- Wyznaczanie strategii na podstawie tych funkcji (algorytm zachłanny, ϵ-zachłanny).

Nowe podejście:

- Wyznaczanie strategii bezpośrednio:

$$\pi_\theta(s) = p(a|s,\theta)$$

Głębokie uczenie ze wzmocnieniem

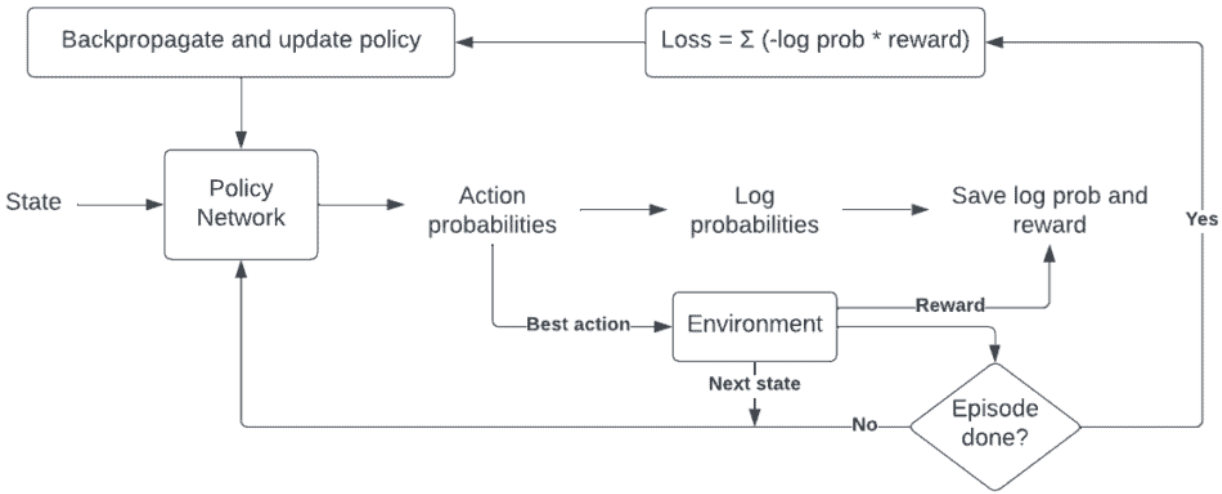
Policy Gradients

Zalety optymalizacji strategii:

- Strategia wyznaczana przez metody Q-Learning jest zawsze deterministyczna. Metody te nie mogą opracować strategii stochastycznej, co może być przydatne w niektórych środowiskach. W takim przypadku wykorzystywane są algorytmy, np. ϵ-zachłanne, ale to podejście nie zawsze jest wystarczające i optymalne.
- Metody należące do grupy optymalizujących strategie mogą być zastosowane w przypadku środowisk, gdzie akcje mają charakter ciągły.
- W sposób ciągły poprawiana jest strategia, a w przypadku metod Q-Learning aproksymowana jest funkcja wartości, a dopiero na jej podstawie wyznaczana jest strategia.

Głębokie uczenie ze wzmocnieniem

Policy Gradients



Dilith Jayakody - Policy REINFORCE - A Quick Introduction (with Code)

Głębokie uczenie ze wzmocnieniem

Policy Gradients

- Algorytmy optymalizacji strategii należą do dziedziny algorytmów optymalizacyjnych.
- Celem jest znalezienie takich wartości parametrów θ, które będą dawały najwyższy wynik funkcji $J(\theta)$.

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] \tag{21}$$

Głębokie uczenie ze wzmocnieniem

Policy Gradients

$$\theta_{t+1} = \theta_t + \alpha \nabla \pi_{\theta_t}(s, a^*)$$

Głębokie uczenie ze wzmocnieniem

Policy Gradients

$$\theta_{t+1} = \theta_t + \alpha \nabla \pi_{\theta_t}(s, a^*)$$

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \nabla \pi_{\theta_t}(s, a)$$

Głębokie uczenie ze wzmocnieniem

Policy Gradients

$$\theta_{t+1} = \theta_t + \alpha \nabla \pi_{\theta_t}(s, a^*)$$

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \nabla \pi_{\theta_t}(s, a)$$

$$\theta_{t+1} = \theta_t + \alpha \frac{\hat{Q}(s, a) \nabla \pi_{\theta_t}(s, a)}{\pi_\theta(s, a)}$$

Głębokie uczenie ze wzmocnieniem

Policy Gradients

$$\theta_{t+1} = \theta_t + \alpha \nabla \pi_{\theta_t}(s, a^*)$$

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \nabla \pi_{\theta_t}(s, a)$$

$$\theta_{t+1} = \theta_t + \alpha \frac{\hat{Q}(s, a) \nabla \pi_{\theta_t}(s, a)}{\pi_\theta(s, a)}$$

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \nabla_\theta log \pi_{\theta_t}(s, a)$$

Głębokie uczenie ze wzmocnieniem

Monte-Carlo Policy Gradient (REINFORCE)

Aktualizacja parametrów metodą gradientu prostego (ang. *stochastic gradient ascent*).

Wykorzystanie nagrody skumulowanej jako wartości funkcji $Q(\hat{s}, a)$.

$$\Delta\theta_t = \alpha G_t \nabla_\theta log \pi_{\theta_t}(s, a)$$

Głębokie uczenie ze wzmocnieniem

Monte-Carlo Policy Gradient (REINFORCE)

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s,\theta)$.
Algorithm parameter: step size $\alpha > 0$.
Initialize policy parameter $\theta \in \mathbb{R}$.
For each episode:

Generate an episode $s_0, a_0, r_1, ..., s_{t-1}, a_{t-1}, r_t$, following $\pi(.|.,\theta)$.
Loop for each step of the episode $t = 0, 1, ..., T-1$:

$$G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$$

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta log \pi_\theta(a_t, s_t|\theta)$$

Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.

Głębokie uczenie ze wzmocnieniem

Trust Region Policy Optimization

Trust Region Policy Optimization (TRPO) is a policy optimization algorithm that improves policy stability by ensuring small and safe updates to the policy.

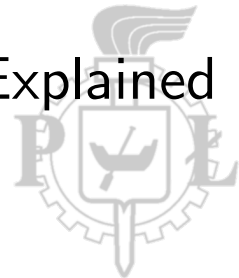
Głębokie uczenie ze wzmocnieniem

Trust Region Policy Optimization

Line search
(like gradient ascent)

Trust region

Jonathan Hui - RL — Trust Region Policy Optimization (TRPO) Explained



Głębokie uczenie ze wzmocnieniem

Trust Region Policy Optimization

$$\theta_{k+1} = \arg\max_{\theta} \mathbb{E}\left[\frac{\pi_{\theta_{\text{old}}}(a \mid s)}{\pi_{\theta}(a \mid s)} A(s,a)\right] \tag{22}$$

$$D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\theta_{\text{old}}}) \leqslant \delta \tag{23}$$

D_{KL} - Kullback–Leibler divergence

Głębokie uczenie ze wzmocnieniem

Trust Region Policy Optimization

1. Collect Trajectories
2. Compute the Advantage Function (Estimate how much better each action is compared to the baseline (value function).
3. Solve the Constrained Optimization Problem
4. Update the Policy Using Natural Gradients

Głębokie uczenie ze wzmocnieniem

Proximal Policy Optimization

The main idea of PPO is to limit the change in the policy by using a clipped objective function. This allows the algorithm to update the policy in a way that keeps the changes "proximal" (i.e., small) without the need for the complex constraints used in TRPO.

Głębokie uczenie ze wzmocnieniem

Proximal Policy Optimization

$$L_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right) \right] \tag{24}$$

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{25}$$

A_t - is the advantage at time step t.

ϵ - is a small clipping parameter, usually between 0.1 and 0.2.

Głębokie uczenie ze wzmocnieniem

Proximal Policy Optimisation

Proximal Policy Optimisation, for estimating $\pi_\theta \approx \pi_*$

Initialize the policy network parameters θ

For each episode:

Collect trajectories using the current policy π_θ

Compute the advantages A_t for each timestep

Compute the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta old}(a_t|s_t)}$

Compute the PPO objective:

$L_{ppo}(\theta) = E_t[min(r_t(\theta) * A_t, clip(r_t(\theta), 1-\epsilon, 1+\epsilon) * A_t)]$

Update θ by maximizing L_{ppo} using gradient ascent

Set $\theta_{old} = \theta$ (old policy becomes the new policy)

Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.

Głębokie uczenie ze wzmocnieniem

REINFORCE with Baseline

$$\theta_{t+1} = \theta_t + \alpha(G_t - b(s_t))\nabla_\theta log\pi_{\theta_t}(s_t, a_t) \tag{26}$$

Metody wyznaczania wartości odniesienia:

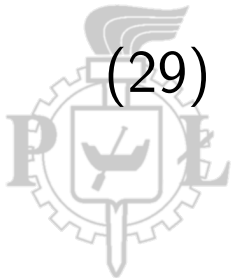
- *whitening*:

$$G_t^* = \frac{G_t - \mu_G}{\sigma_G} \tag{27}$$

- *learned baseline*:

$$\theta_{t+1} = \theta_t + \alpha(G_t - \hat{v}(s_t, w))\nabla_\theta log\pi_{\theta_t}(s_t, a_t) \tag{28}$$

$$w_{t+1} = w_t + \beta(G_t - \hat{v}(s_t, w))\nabla_v(s_t, w) \tag{29}$$



Głębokie uczenie ze wzmocnieniem

REINFORCE with Baseline

REINFORCE with Baseline (episodic), for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\theta)$.
Input: a differentiable state-value function parameterization $\upsilon(s,w)$.
Algorithm parameter: step sizes $\alpha > 0$ and $\beta > 0$.
Initialize policy parameter $\theta \in \mathbb{R}$ and state-value weights $w \in \mathbb{R}$.
For each episode:

Generate an episode $s_0, a_0, r_1, ..., s_{t-1}, a_{t-1}, r_t$, following $\pi(.|.,\theta)$.
Loop for each step of the episode $t = 0, 1, ..., T-1$:

$$G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$$
$$\delta \leftarrow G - \hat{\upsilon}(s_t, w)$$
$$w \leftarrow w + \beta\delta\nabla_w \upsilon(s_t, w)$$
$$\theta \leftarrow \theta + \alpha\delta\nabla_\theta log\pi_\theta(a_t, s_t|\theta)$$

Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.

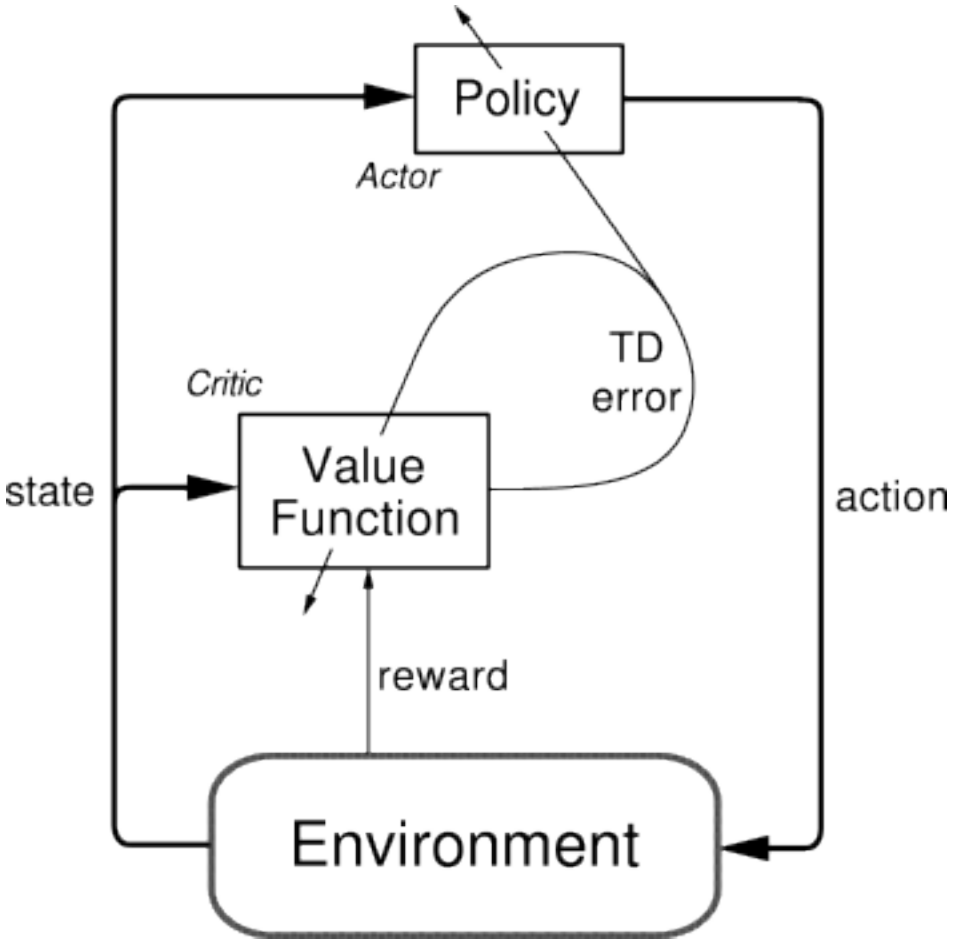
Głębokie uczenie ze wzmocnieniem

Actor-Critic

- uczy się strategii - $\pi_\theta(s, a)$,
- uczy się funkcji oceny stanu - $V_\theta(s)$,
- wykorzystać $V_\theta(s)$, żeby szybciej nauczyć się strategii $\pi_\theta(s, a)$.

Głębokie uczenie ze wzmocnieniem

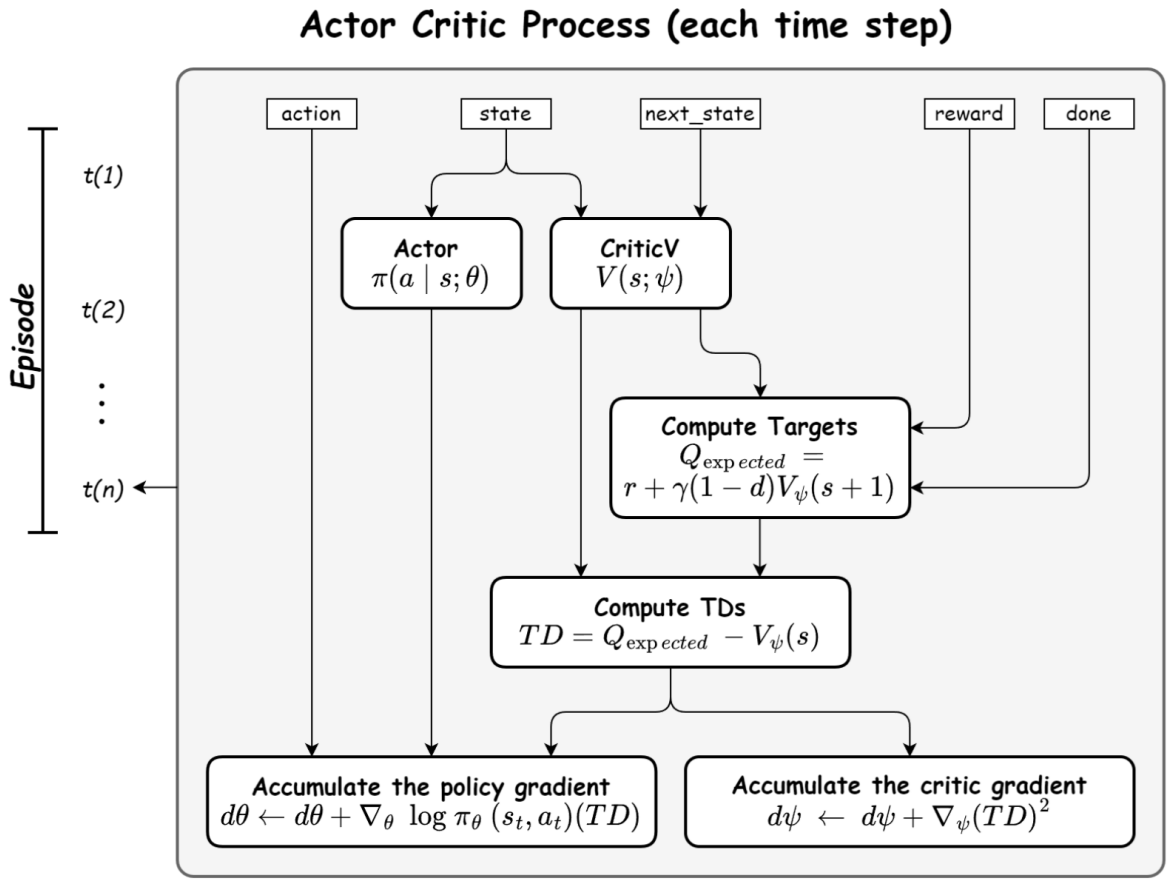
Actor Critic



Sutton, Richard S., and Andrew G. Barto. Reinforcement learning: An introduction. Vol. 1. No. 1. Cambridge: MIT press, 1998.

Głębokie uczenie ze wzmocnieniem

Actor Critic



Deep Learning Bible - 5. Reinforcement Learning - Eng.

Głębokie uczenie ze wzmocnieniem

Actor-Critic

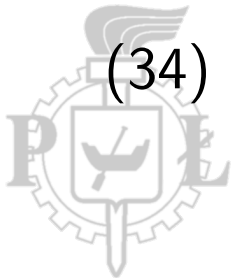
$$\theta_{t+1} = \theta_t + \alpha(G_{t:t+1} - \hat{v}(s_t, w))\nabla_\theta log\pi_\theta(a_t, s_t|\theta) \quad (30)$$

$$= \theta_t + \alpha(r_{t+1} + \gamma\hat{v}(s_{t+1}, w) - \hat{v}(s_t, w))\nabla_\theta log\pi_\theta(a_t, s_t|\theta) \quad (31)$$

$$= \theta_t + \alpha\delta_t\nabla_\theta log\pi_\theta(a_t, s_t|\theta) \quad (32)$$

$$\delta_t = r_{t+1} + \gamma\hat{v}(s_{t+1}, w) - \hat{v}(s_t, w) \quad (33)$$

$$w_{t+1} = w_t - \alpha_{\text{critic}} \cdot \nabla_w \left[\delta_t^2\right] \quad (34)$$



Głębokie uczenie ze wzmocnieniem

Actor-Critic

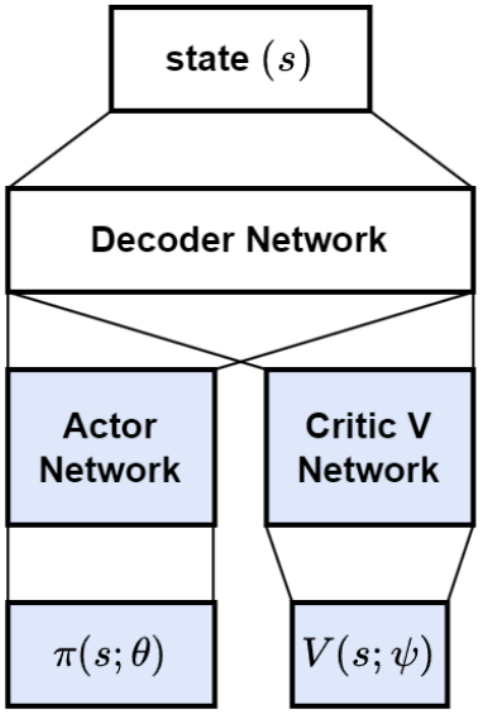
One-step Actor-Critic, for estimating $\pi_\theta \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a|s,\theta)$.
Input: a differentiable state-value function parameterization $\upsilon(s,w)$.
Algorithm parameter: step sizes $\alpha > 0$ and $\beta > 0$.
Initialize policy parameter $\theta \in \mathbb{R}$ and state-value weights $w \in \mathbb{R}$.
For each episode:
    Initialize s (first step of episode).
    Loop while s is not terminal (for each time step):
        Choose action a according to the policy $\pi_\theta(.,s,\theta)$,
        Take action a, observe s', r,
        $\delta \leftarrow r + \gamma\hat{\upsilon}(s',w) - \hat{\upsilon}(s,w)$.
        $w \leftarrow w + \beta\delta\nabla_w\upsilon(s',w)$.
        $\theta \leftarrow \theta + \alpha\delta\nabla_\theta log\pi_\theta(a,s|\theta)$.
        $s \leftarrow s'$.

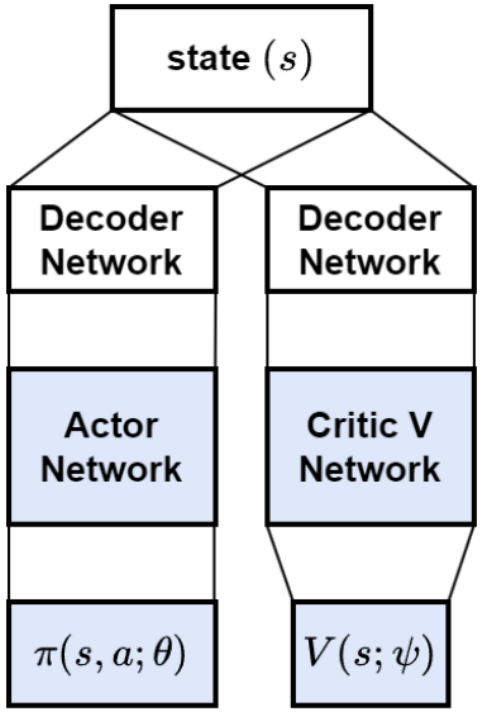
Richard S. Sutton and Andrew G. Barto. Reinforcement learning: An introduction. MIT press, 2018.

Głębokie uczenie ze wzmocnieniem

Actor Critic



Two-head Actor Critic



Decoupled Actor Critic

Deep Learning Bible - 5. Reinforcement Learning - Eng.

Głębokie uczenie ze wzmocnieniem

Advantage Actor-Critic (A2C)

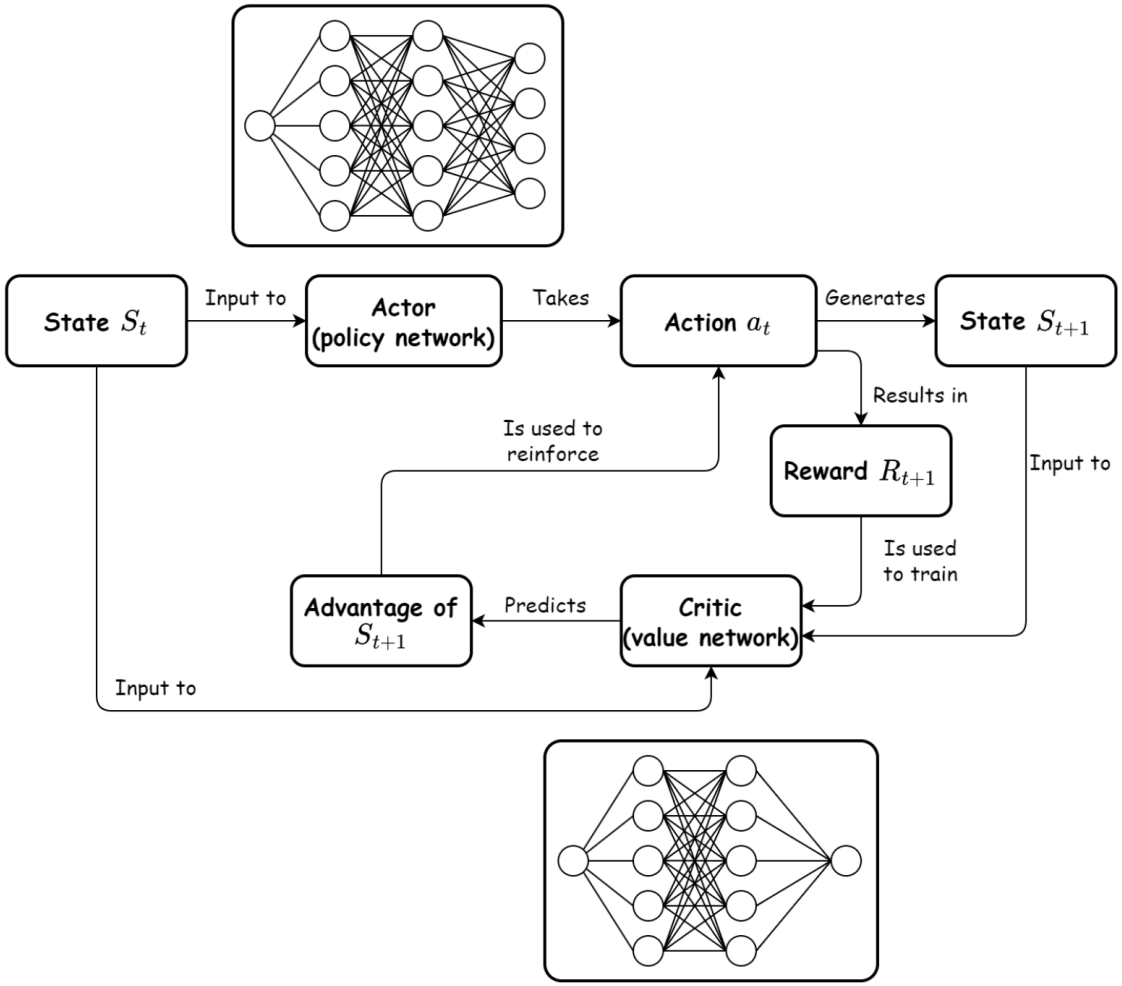
Zmiana uczenia w przypadku Actor Network - wykorzystanie Advantage

$$A(s_t, a_t) = R_t - V(s_t) \tag{35}$$

$$\theta_{\text{actor}} \leftarrow \theta_{\text{actor}} + \alpha_{\text{actor}} \cdot \nabla_{\theta_{\text{actor}}} [\log \pi_\theta(a_t \mid s_t) \cdot (R_t - V(s_t))] \tag{36}$$

Głębokie uczenie ze wzmocnieniem

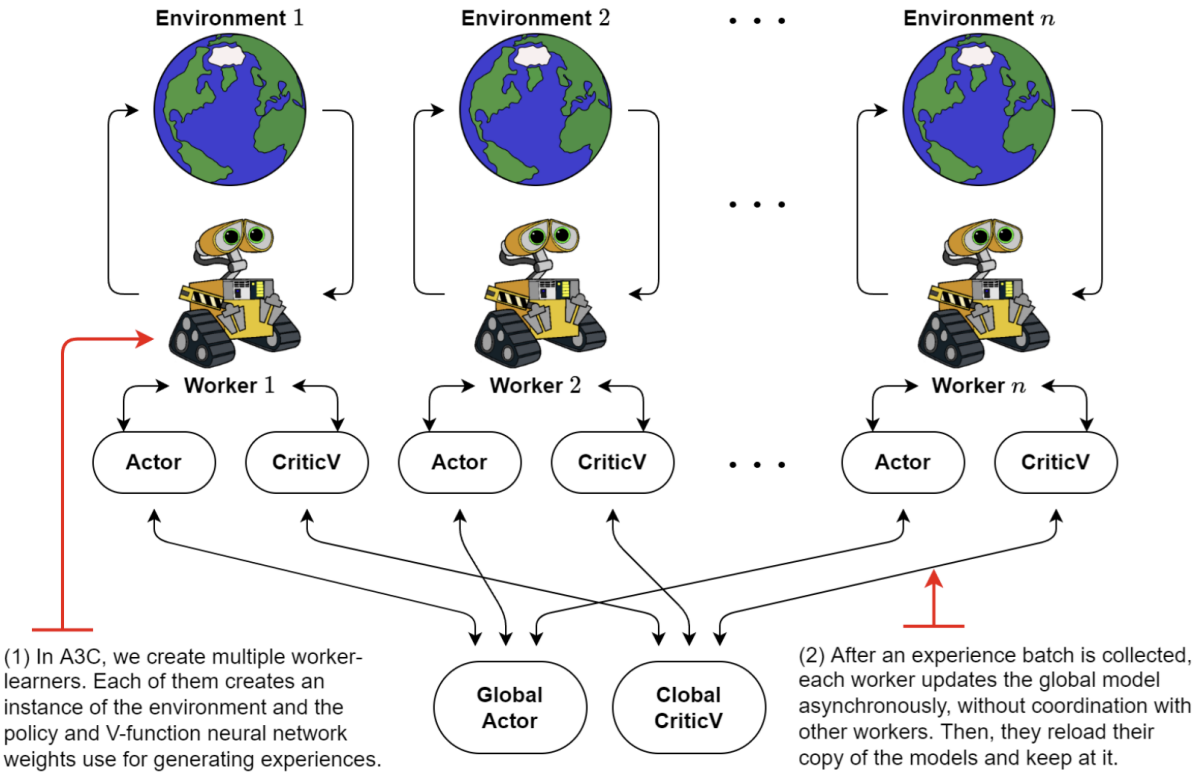
Advantage Actor-Critic (A2C)



Deep Learning Bible - 5. Reinforcement Learning - Eng.

Głębokie uczenie ze wzmocnieniem

Asynchronous Advantage Actor-Critic (A3C)



Deep Learning Bible - 5. Reinforcement Learning - Eng.

Głębokie uczenie ze wzmocnieniem

Asynchronous Advantage Actor-Critic (A3C)

1. Initialize Global Model
2. Spawn Multiple Worker Agents
3. Workers Interact with the Environment
4. Compute Gradients Locally
5. Update Global Model
6. Sync Local Model with Global Model

Głębokie uczenie ze wzmocnieniem

Asynchronous Advantage Actor-Critic (A3C)

- Each worker computes the Advantage function:

$$A(s_t, a_t) = R_t - V(s_t)$$

- The actor loss (policy gradient update) is computed as:

$$L_{\text{actor}} = -\log \pi_\theta(a_t \mid s_t) \cdot A(s_t, a_t)$$

- The critic loss (value function update) is computed using Mean Squared Error (MSE):

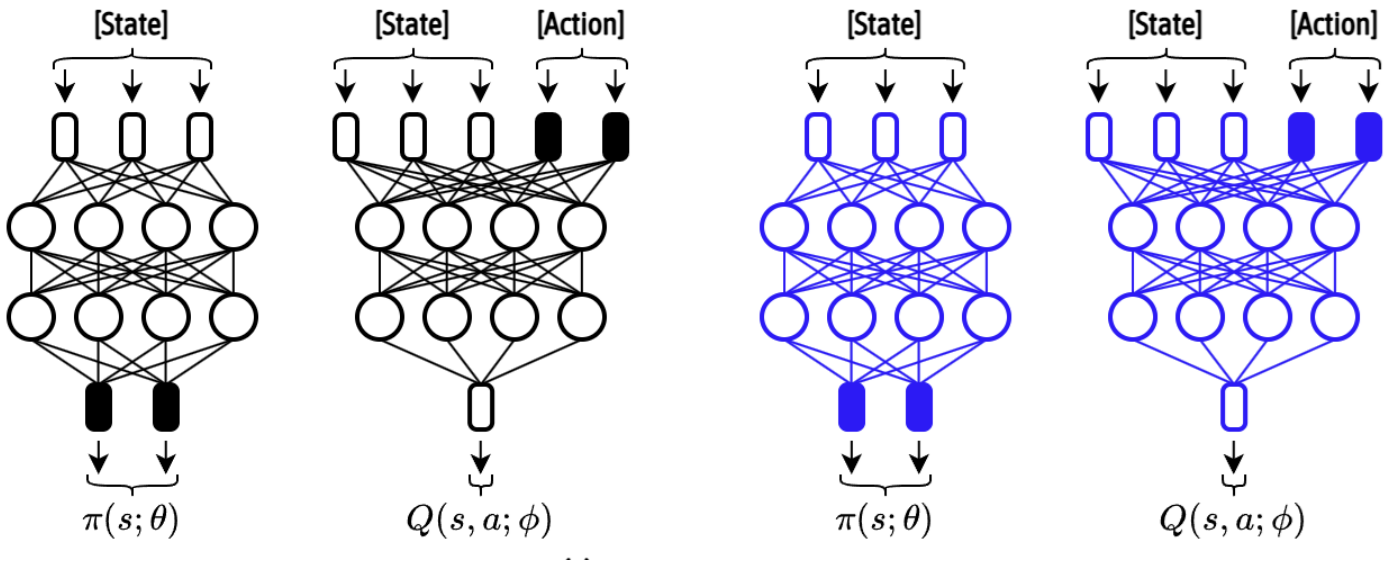
$$L_{\text{critic}} = (R_t - V(s_t))^2$$

- The entropy loss is added to encourage exploration:

$$L_{\text{entropy}} = -\sum \pi_\theta \log \pi_\theta$$

Głębokie uczenie ze wzmocnieniem

Deep Deterministic Policy Gradient (DDPG)



Deep Learning Bible - 5. Reinforcement Learning - Eng.

Głębokie uczenie ze wzmocnieniem

Deep Deterministic Policy Gradient (DDPG)

1. Actor Network $\mu_\theta(s)$: Maps states s to actions a and is updated via policy gradient.
2. Critic Network $Q_\theta(s, a)$: Estimates the Q-value of a given state-action pair $Q(s, a)$, helping the actor improve.
3. Experience Replay Buffer: Stores past experiences (s, a, r, s') to break correlation between updates.
4. Target Networks: Helps stabilize training by using a slowly updating copy of the actor and critic networks.

Głębokie uczenie ze wzmocnieniem

Deep Deterministic Policy Gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG)

Initialize Actor Network $\mu_\theta(s)$ and Critic Network $Q_\phi(s, a)$ with random weights.
Initialize Target Actor Network $\mu_{\theta'}(s)$ and Target Critic Network $Q_{\phi'}(s, a)$ as copies of the original networks.
Initialize Replay buffer $\mathcal{D}$.
For each episode:

- Start with an initial state s_0.
- Select an action using the Actor Network: $a_t = \mu_\theta(s_0) + \mathcal{N}_t$.
- Execute action a_t, observe reward r_t and new state s_{t+1}.
- Store experience (s_t, a_t, r_t, s_{t+1}) in the replay buffer.

Sample a mini-batch (s_t, a_t, r_t, s_{t+1}) from the replay buffer:

- Update the critic $Q_\phi(s, a)$.
  - Compute target Q-value: $y = r_t + Q_{\phi'}(s', \mu_{\theta'}(s'))$.
  - Minimize critic loss: $L_{critic} = \mathcal{E}[y - Q_\phi(s, a)^2]$.
  - Update critic network by gradient descent.

Głębokie uczenie ze wzmocnieniem

Deep Deterministic Policy Gradient (DDPG)

Deep Deterministic Policy Gradient (DDPG)

Update the actor $\mu_\theta(s)$.

Compute the policy gradient: $\nabla_\theta J \approx \mathbb{E}\left[\nabla_a Q_\phi(s,a)\Big|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)\right]$.

Update the actor network using gradient ascent.

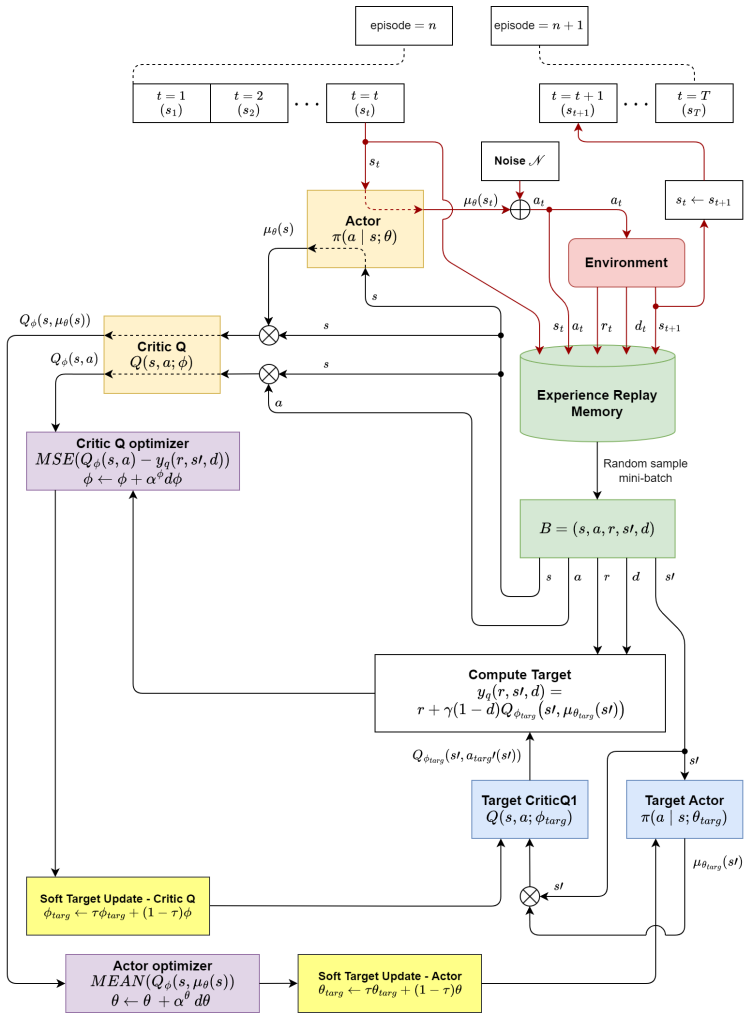
Update target networks:

Slowly update the target networks using:

$$\theta' \leftarrow \tau\theta + (1-\tau)\theta'$$
$$\phi' \leftarrow \tau\phi + (1-\tau)\phi'$$

Głębokie uczenie ze wzmocnieniem

Deep Deterministic Policy Gradient (DDPG)



Deep Learning Bible - 5. Reinforcement Learning - Eng.

Głębokie uczenie ze wzmocnieniem

Materiały uzupełniające

- Książka *Reinforcement Learning: An Introduction*, Richard S. Sutton and Andrew G. Barto, wydanie drugie, 2018.
  - Rozdziały 13.1 - 13.4 - *Policy Gradient Methods*.
  - Rozdziały 13.5 - *Actor–Critic Methods*.
- Video *Hado Van Hasselt, Advanced Deep Learning & Reinforcement Learning Lectures. Reinforcement Learning 6: Policy Gradients and Actor Critics*.
- Video *RL Course by David Silver - Lecture 7: Policy Gradient Methods*.

Głębokie uczenie ze wzmocnieniem

Imitation Learning

- Exploration is Dangerous or Expensive
- Reward Functions are Hard to Define
- Faster Convergence is Needed
- The Task is Already Solved by Experts

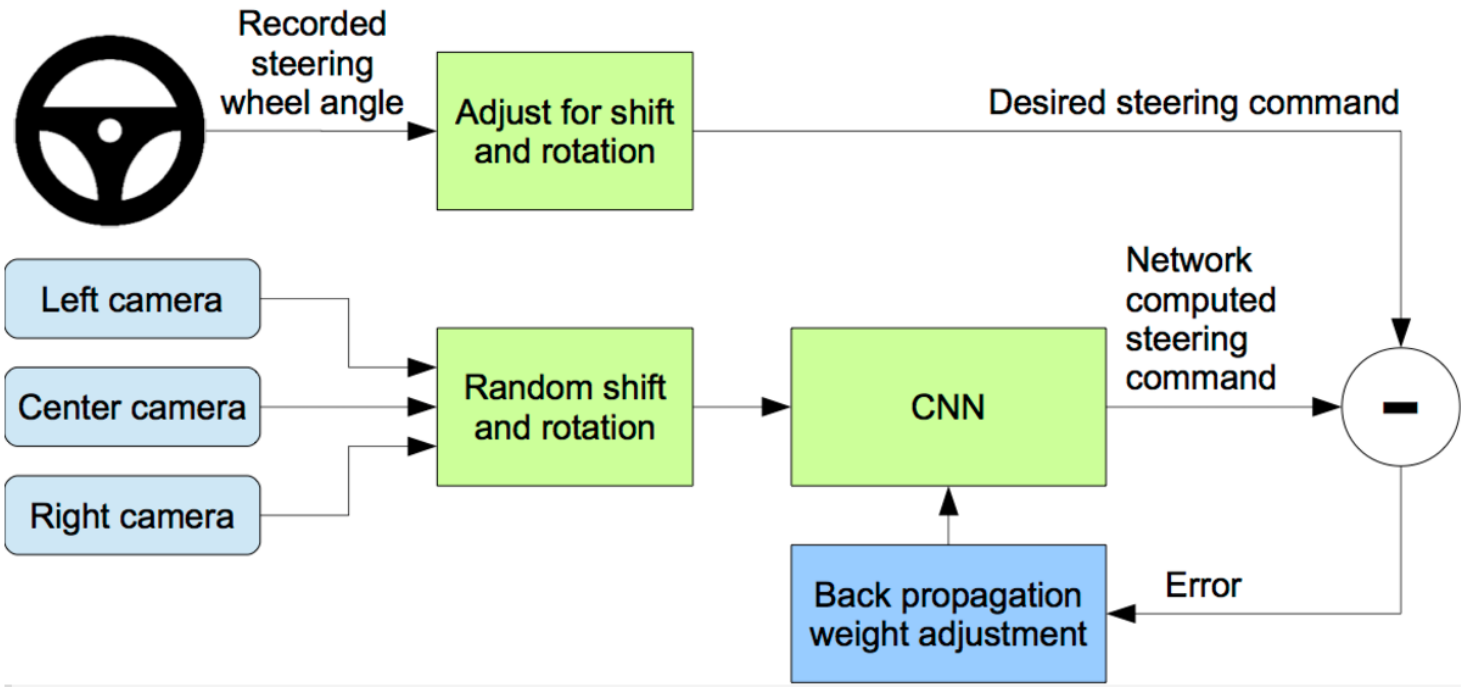
Głębokie uczenie ze wzmocnieniem

Imitation Learning

- Exploration is Dangerous or Expensive
- Reward Functions are Hard to Define
- Faster Convergence is Needed
- The Task is Already Solved by Experts
- No Expert Demonstrations are Available
- The Expert is Suboptimal or Noisy
- Generalization is Required

Głębokie uczenie ze wzmocnieniem

Behavioral Cloning



Behaviour Cloning (Learning Driving Pattern) — CarND - Satyasheel

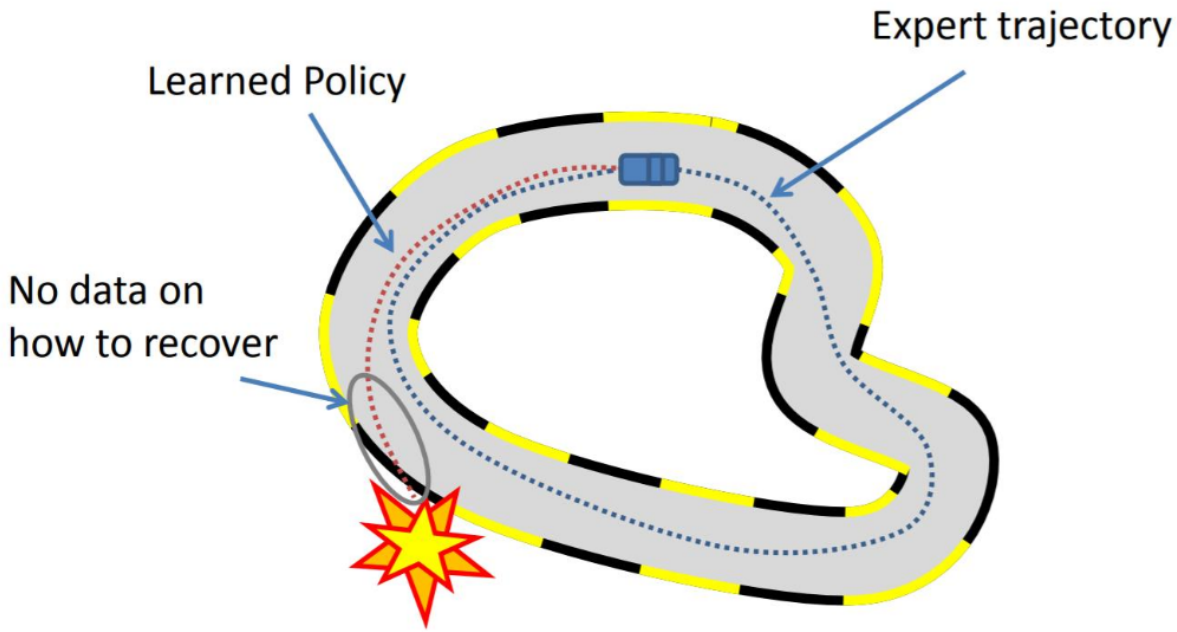
Głębokie uczenie ze wzmocnieniem

Behavioral Cloning

1. Collect Expert Demonstrations - A human or pre-trained expert agent plays the task and generates a dataset of states and actions.
2. Train a Policy $\pi_\theta(a|s)$.
   - Supervised Learning problem.
   - Minimize the difference between the expert's actions and the learned policy's actions.
   - Loss function: $L(\theta) = \mathbb{E}_{(s,a)\sim D}\left[\|\pi_\theta(s) - a\|^2\right]$

Głębokie uczenie ze wzmocnieniem

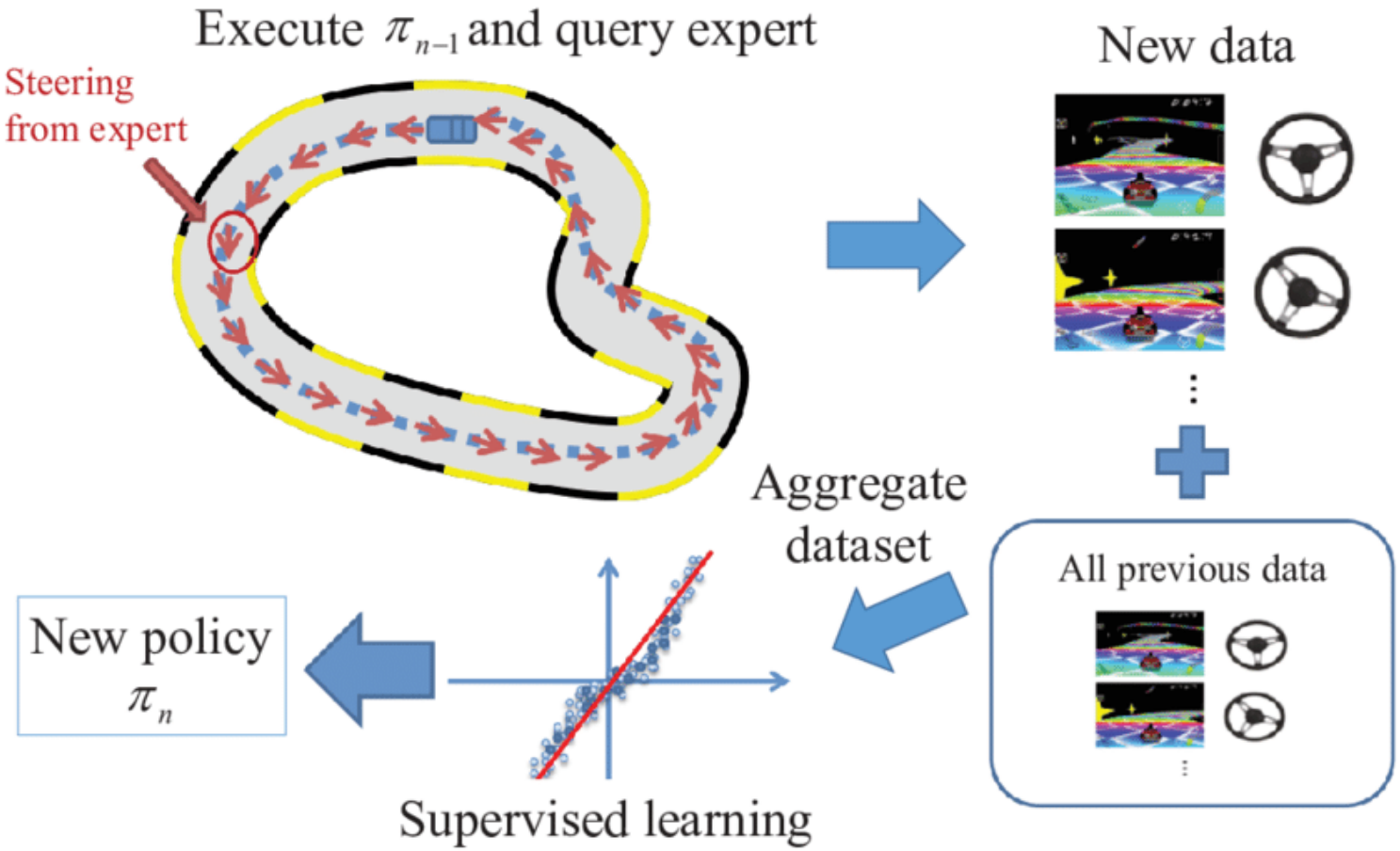
Behavioral Cloning



Introduction to DAgger - Shuijing Liu

Głębokie uczenie ze wzmocnieniem

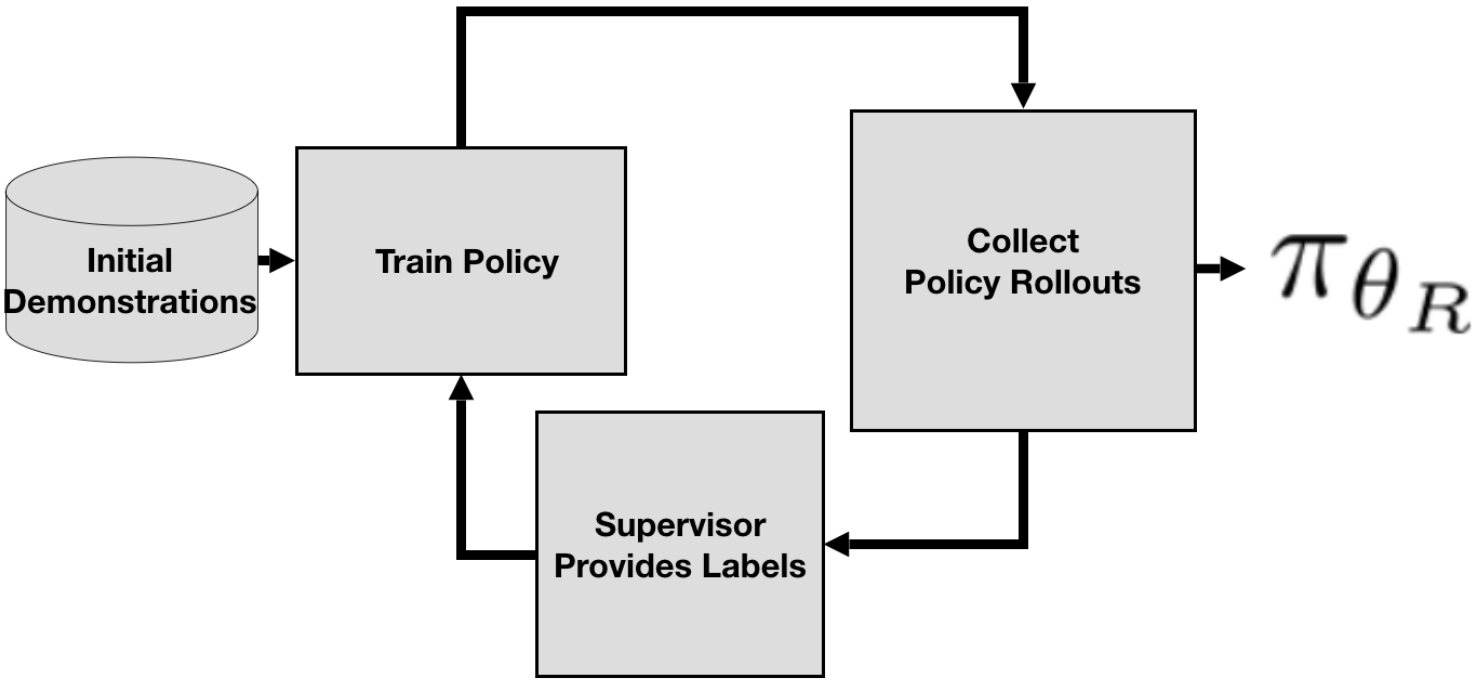
DAgger



An Invitation to Imitation - J. Andrew Bagnell

Głębokie uczenie ze wzmocnieniem

DAgger



Głębokie uczenie ze wzmocnieniem

DAgger

1. Train Initial Policy - behavior cloning policy π_θ using expert demonstrations $D_0 = \{(s, a)\}$.
2. Deploy and Collect New Data - The expert observes and corrects the agent's actions, providing new demonstrations for visited states.
3. Aggregate Data - Combine the new expert demonstrations with the existing dataset $D \leftarrow D \cup D_{\text{new}}$.
4. Retrain π_θ on the updated dataset.
5. Repeat Until Convergence.

Głębokie uczenie ze wzmocnieniem

AggreVaTe: Aggregate Values to Imitate

1. Train Initial Policy π_0 - behavior cloning policy π_θ using expert demonstrations $D_0 = \{(s, a)\}$.
2. Roll Out the Policy π_t.
3. Compute the Cost-to-Go for Each Action - Instead of just asking the expert for the correct action, we estimate the long-term cost if we take a certain action.
4. Aggregate Expert-Labeled Data.
5. Retrain π_θ on the updated dataset.
6. Repeat Until Convergence.

Głębokie uczenie ze wzmocnieniem

AggreVaTe: Aggregate Values to Imitate

1. Start at State s, Take Action a (Exploration Step).
2. Follow the Expert Policy $\pi*$ for the Rest of the Episode.
3. Sum Up the Observed Costs from t to T:

$$\hat{Q}(s,a) = \sum_{t'=t}^{T} C(s_{t'}, a_{t'}) \tag{37}$$

Głębokie uczenie ze wzmocnieniem

AggreVaTe: Aggregate Values to Imitate

AggreVaTe: Aggregate Values to Imitate

Initialize $\mathcal{D} \leftarrow \emptyset$, $\hat{\pi}_1$ to any policy in Π.
for $i = 1$ to N **do**
    Let $\pi_i = \beta_i \pi^* + (1 - \beta_i)\hat{\pi}_i$
    Collect m data points as follows:
        for $j = 1$ to m **do**
            Sample uniformly $t \in \{1, 2, \ldots, T\}$.
            Start new trajectory in some initial state drawn from initial state distribution.
            Execute current policy π_i up to time $t - 1$.
            Execute some exploration action a_t in current state s_t at time t.
            Execute expert from time $t + 1$ to T, and observe estimate of cost-to-go $\hat{Q}$ starting at time t.
        end for
        Get dataset $\mathcal{D}_i = \{(s, t, a, \hat{Q})\}$ of states, times, actions, with expert's cost-to-go.
        Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_i$.
        Train cost-sensitive classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$.
    end for
    Return best $\hat{\pi}_i$ on validation.

Głębokie uczenie ze wzmocnieniem

1. Środowisko.
2. Nagroda.
3. Stan.
4. Akcja.
5. Strategia.

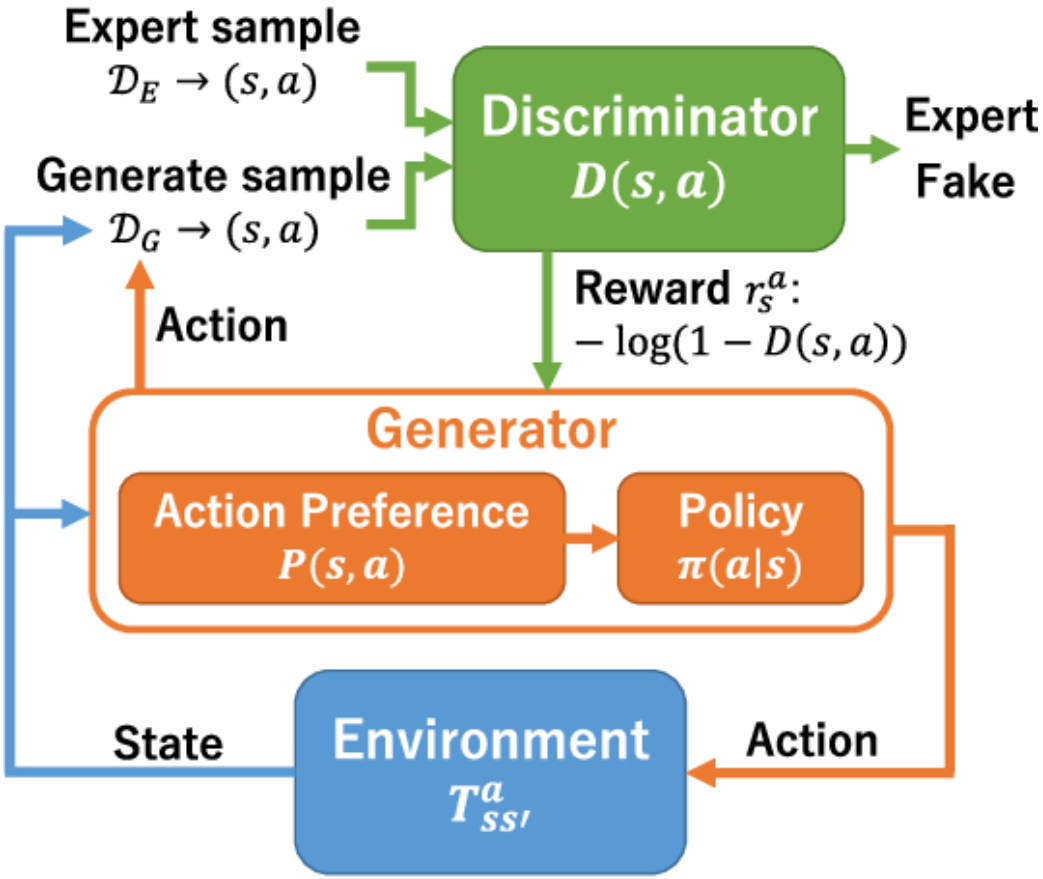
Głębokie uczenie ze wzmocnieniem

Inverse Reinforcement Learning

1. Collect Expert Demonstrations.
2. Model the Expert's Policy.
3. Infer the Reward Function $R(s)$.
4. Train an RL Agent on the Learned Reward Function.

Głębokie uczenie ze wzmocnieniem

Generative Adversarial Imitation Learning



Tsurumine, Y., Cui, Y., Yamazaki, K., & Matsubara, T. (2019, October). Generative adversarial imitation learning with deep p-network for robotic cloth manipulation. In 2019 IEEE-RAS 19th International Conference on Humanoid Robots (Humanoids) (pp. 274-280). IEEE.

Głębokie uczenie ze wzmocnieniem

Generative Adversarial Imitation Learning

1. Directly extracting a policy from data as if it were obtained by RL following IRL.
2. Bypassing any intermediate IRL step.
3. Draws an analogy between imitation learning and generative adversarial networks (GAN).
4. Derive a model-free between imitation learning algorithm with significant performance improvement with low sample and computational complexity.

Głębokie uczenie ze wzmocnieniem

Generative Adversarial Imitation Learning

- Discriminator $D(s, a)$: Tries to distinguish expert demonstrations from the policy's actions.
- Generator (Policy) $\pi_\theta(a|s)$: Tries to fool the discriminator into thinking its actions are expert-like.

Głębokie uczenie ze wzmocnieniem

Generative Adversarial Imitation Learning

1. The policy π_θ generates actions in the environment.
2. The discriminator $D(s, a)$ assigns a probability of whether these actions are from the expert or the agent.
3. The policy is updated using reinforcement learning (usually TRPO or PPO) to maximize the probability of fooling the discriminator.
4. The discriminator is updated using binary classification loss (cross-entropy) to differentiate between expert and agent actions.

Głębokie uczenie ze wzmocnieniem

Generative Adversarial Imitation Learning

- Discriminator Loss (Binary classification between expert and policy actions):

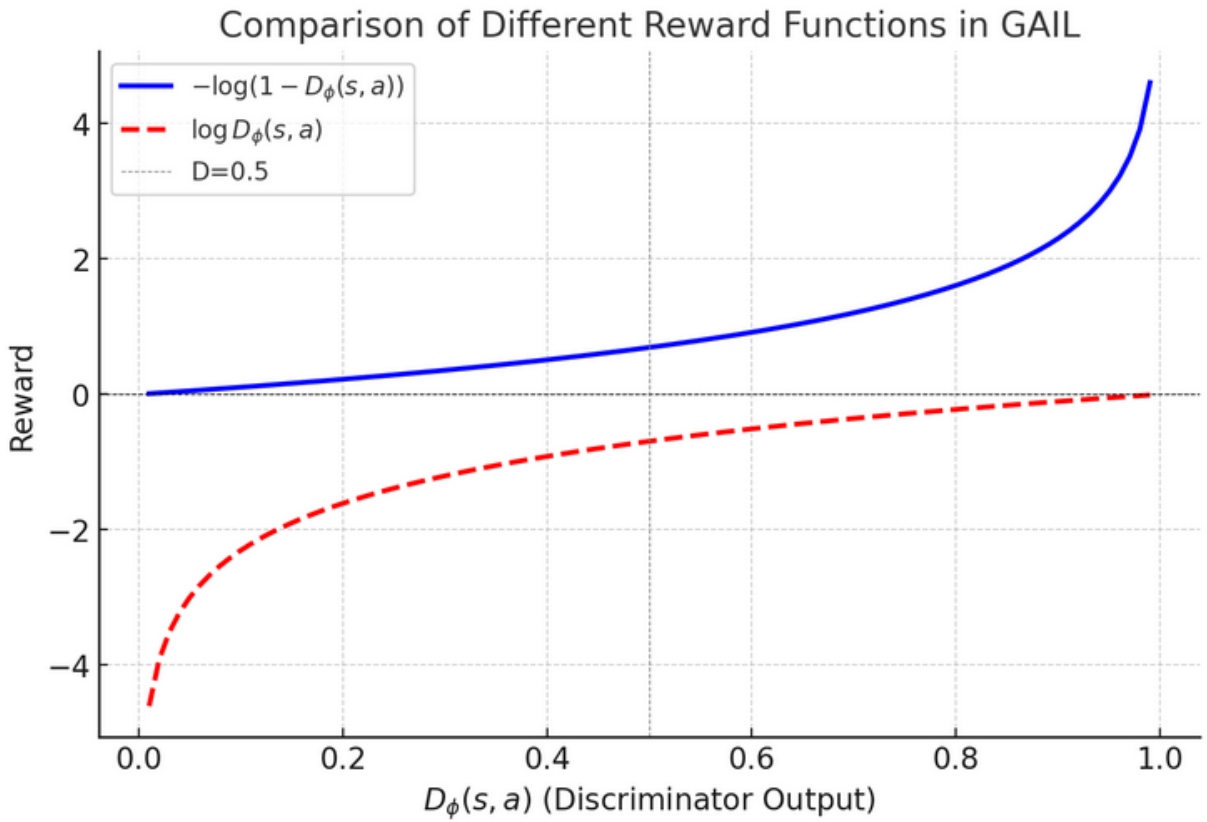
$$\mathcal{L}_{\mathcal{D}} = -\mathbb{E}_{(s,a)\sim\pi_{\text{expert}}}\left[\log D(s,a)\right] - \mathbb{E}_{(s,a)\sim\pi}\left[\log(1 - D(s,a))\right] \quad (38)$$

- Policy Update:

$$\mathcal{L}_{\pi} = \mathbb{E}_{(s,a)\sim\pi_{\theta}}\left[-\log D(s,a)\right] \quad (39)$$

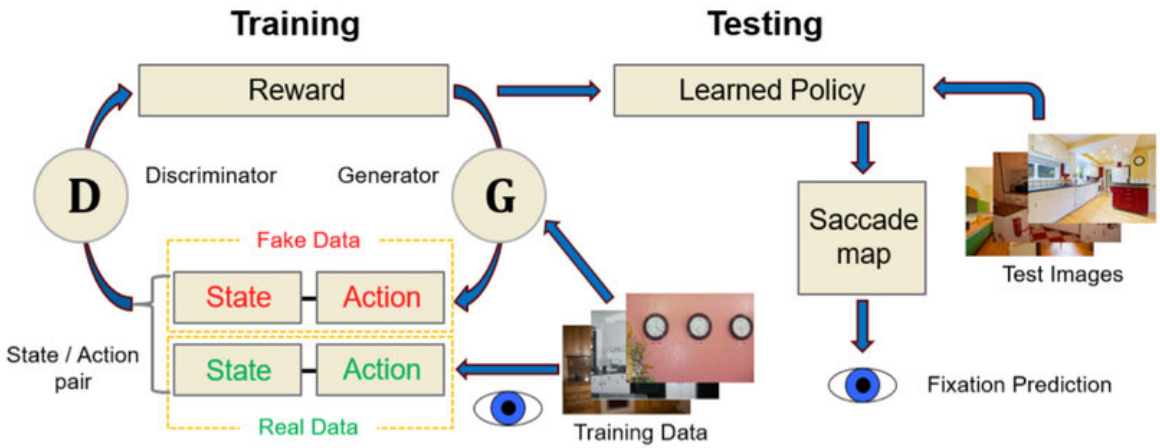
Głębokie uczenie ze wzmocnieniem

Generative Adversarial Imitation Learning



Głębokie uczenie ze wzmocnieniem

Adversarial Inverse Reinforcement Learning



Zelinsky, G. J., Chen, Y., Ahn, S., Adeli, H., Yang, Z., Huang, L., ... & Hoai, M. (2021). Predicting goal-directed attention control using inverse-reinforcement learning. Neurons, behavior, data analysis and theory, 2021, 10-51628.

Głębokie uczenie ze wzmocnieniem

Adversarial Inverse Reinforcement Learning

Adversarial Inverse Reinforcement Learning

Obtain expert trajectories τ_i^E
Initialize policy π and discriminator $D_{\theta,\phi}$
for step t in $1...N$ **do**
    Collect trajectories $\tau_i = (s_0, a_0, ..., s_T, a_T)$ by executing π.
    Train $D_{\theta,\phi}$ via binary logistic regression to classify expert data τ_i^E from samples τ_i.
    Update reward $r_{\theta,\phi}(s, a, s') \leftarrow logD_{\theta,\phi}(s, a, s') - log(1 - D_{\theta,\phi}(s, a, s'))$
    Update π with respect to $r_{\theta,\phi}$ using any policy optimization method.
end for

Fu, J., Luo, K., & Levine, S. (2017). Learning robust rewards with adversarial inverse reinforcement learning. arXiv preprint arXiv:1710.11248.

Głębokie uczenie ze wzmocnieniem

Adversarial Inverse Reinforcement Learning

1. Initialization of Neural Networks:
   - Policy Network $\pi_\theta(a \mid s)$.
   - Discriminator $D_\phi(s, a)$ (reward function $f_\phi(s, a)$, potential function g_θ).
2. The discriminator $D(s, a)$:

$$D_\phi(s, a) = \frac{e^{f_\phi(s,a) - g_\theta(s)}}{e^{f_\phi(s,a) - g_\theta(s)} + \pi_\theta(a \mid s)} \tag{40}$$

3. The discriminator is trained using binary cross-entropy loss:

$$\mathcal{L}_\mathcal{D} = -\mathbb{E}_{(s,a)\sim\pi_{\text{expert}}} [\log D(s, a)] - \mathbb{E}_{(s,a)\sim\pi} [\log(1 - D(s, a))] \tag{41}$$

4. Extracting reward function $r_\phi(s, a) = f_\phi(s, a) - \log \pi_\theta(a \mid s)$.
5. Training policy $\mathcal{L}_\pi = \mathbb{E}_{(s,a)\sim\pi_\theta} [-\log D_\phi(s, a)]$

Głębokie uczenie ze wzmocnieniem

Model-Based Reinforcement Learning

Głębokie uczenie ze wzmocnieniem

Dyna-Q

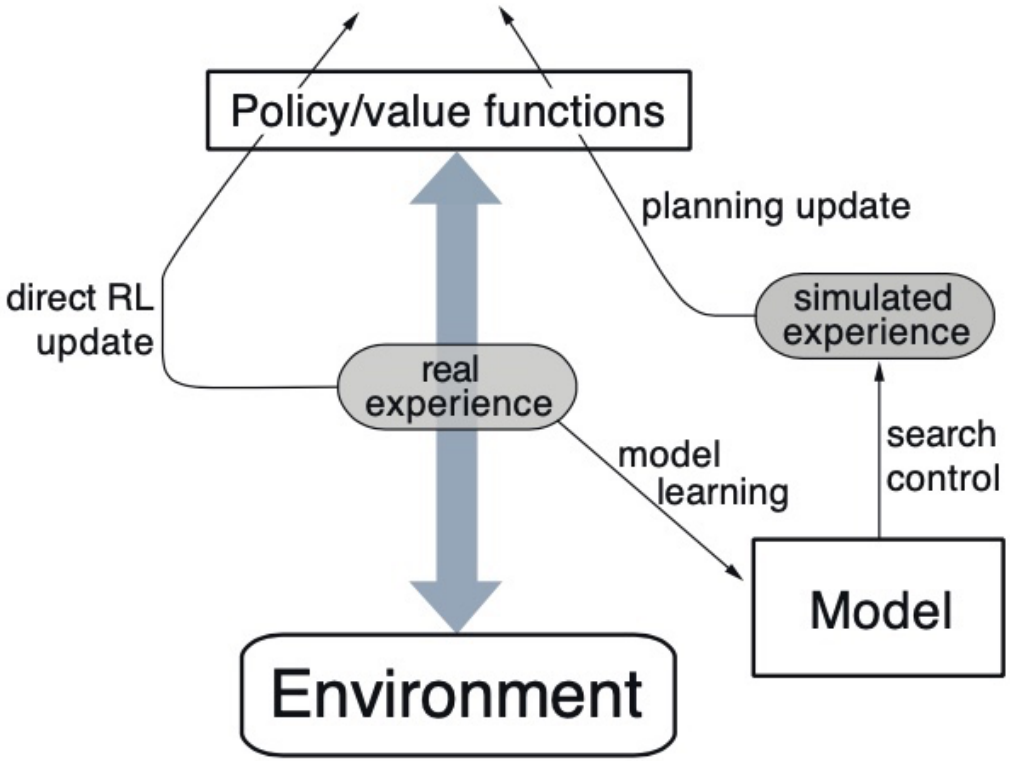


Figure 8.1: The general Dyna Architecture. Real experience, passing back and forth between the environment and the policy, affects policy and value functions in much the same way as does simulated experience generated by the model of the environment.

Notes on AI

Głębokie uczenie ze wzmocnieniem

Dyna-Q

Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

Loop forever:

$S \leftarrow$ current (nonterminal) state

$A \leftarrow \epsilon$-greedy(S,Q)

Take action A; observe resultant reward, R, and state, S'

$$Q(S,A) \leftarrow Q(S,A) + \alpha \left[R + \gamma \max_a Q(S',a) - Q(S,A) \right]$$

$Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)

Loop repeat n **times:**

$S \leftarrow$ random previously observed state

$A \leftarrow$ random action previously taken in S

$R, S' \leftarrow Model(S,A)$

$Q(S,A) \leftarrow Q(S,A) + \alpha \left[R + \gamma \max_a Q(S',a) - Q(S,A) \right]$

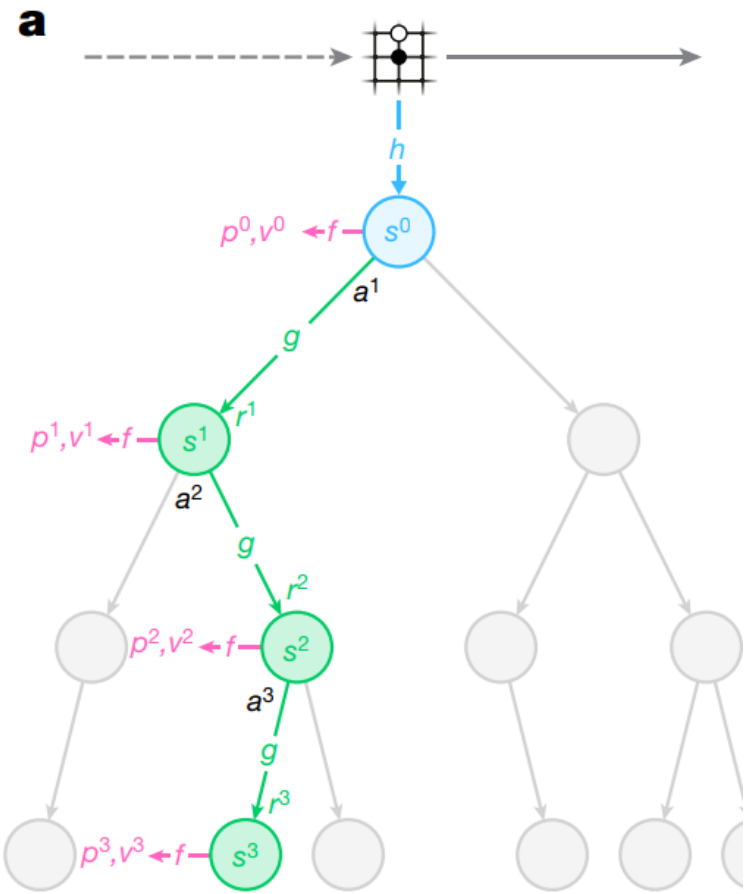
MuZero

MuZero

- *representation function* - h - ResNet, generuje ukryty stan (256 *planes*),
- *prediction function* - f - strategia i oczekiwany wynik,
- *dynamics function* - g - model na podstawie stanu i akcji generuje kolejny stan i nagrodę.

MuZero

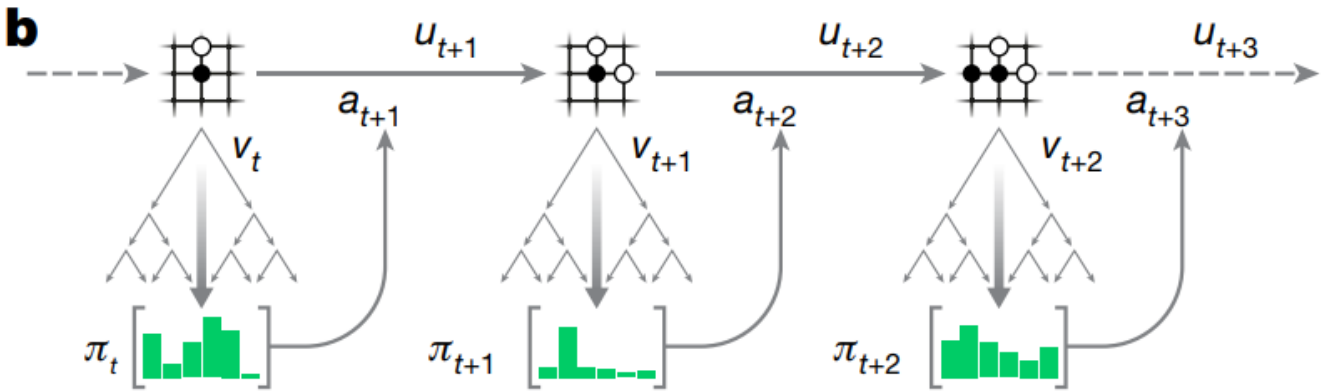
Planowanie



Rysunek 12: MuZero - planowanie

MuZero

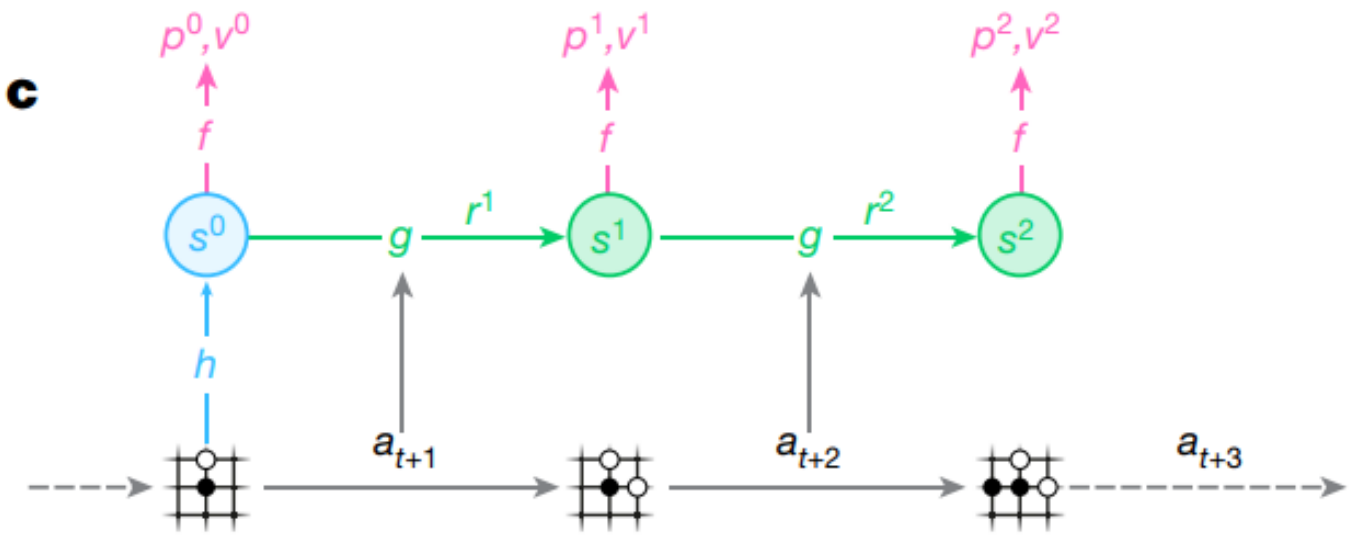
Działanie



Rysunek 13: MuZero - działanie

MuZero

Nauka



Rysunek 14: MuZero - nauka

MuZero

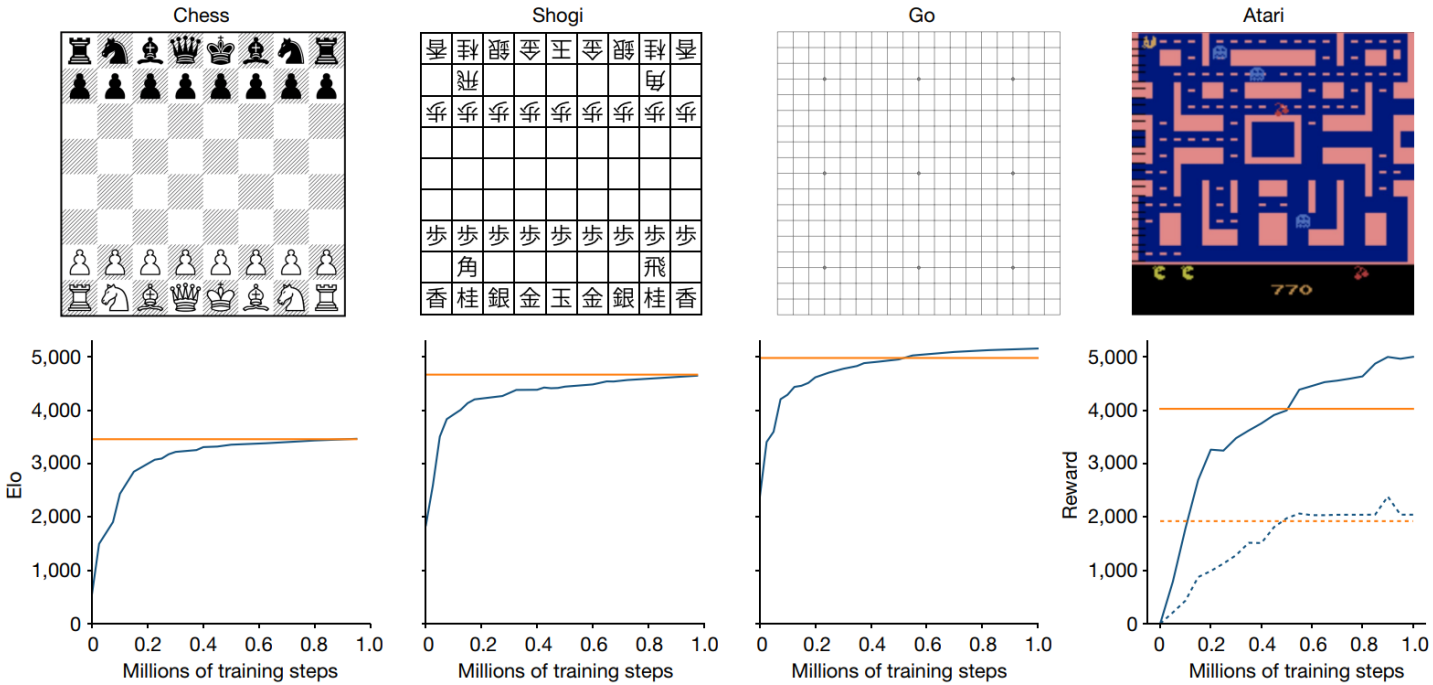
Nauka

$$l_t(\theta) = \sum_{k=0}^{K} l^{\mathrm{p}}(\pi_{t+k}, p_t^k) + \sum_{k=0}^{K} l^{\mathrm{v}}(z_{t+k}, v_t^k) + \sum_{k=1}^{K} l^{\mathrm{r}}(u_{t+k}, r_t^k) + c\|\theta\|^2,$$

Rysunek 15: MuZero - nauka

MuZero

Wyniki



Rysunek 16: MuZero - wynik

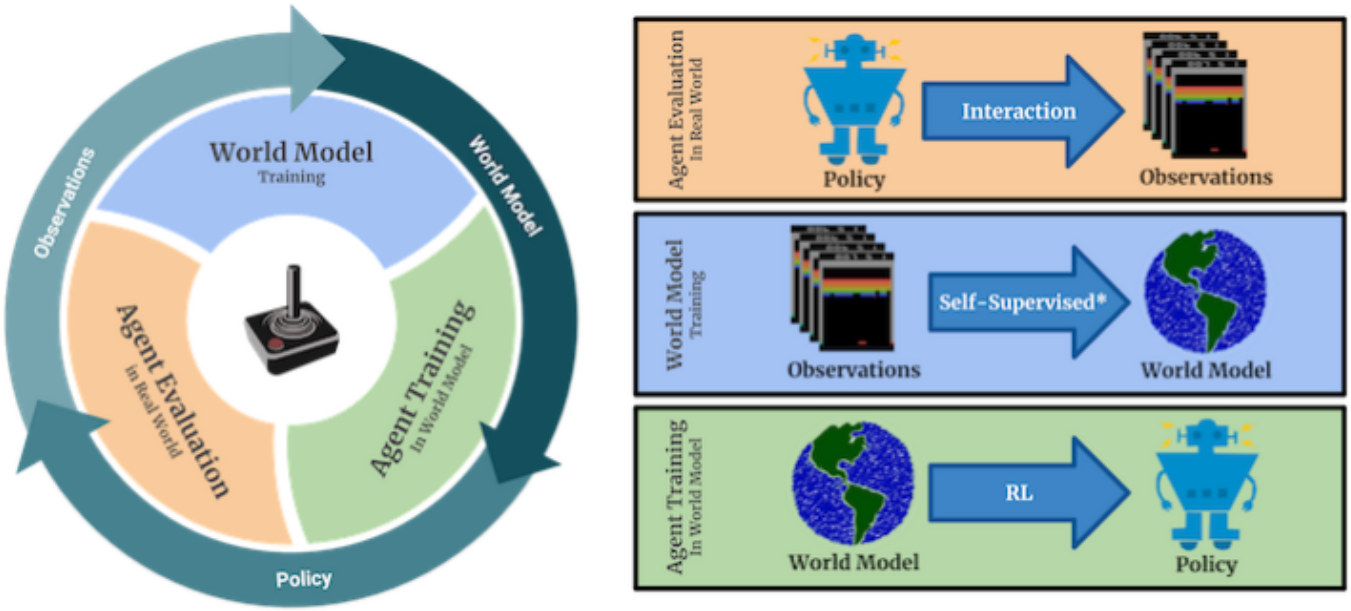
SimPLe

SimPLe

- SimPLE stands for **Simulated Policy Learning**.
- It is a model-based reinforcement learning algorithm designed for high-dimensional inputs (e.g., pixel observations).
- Key idea: Learn a latent world model to *simulate* future states and train a policy with both real and imagined rollouts.

SimPLe

SimPLe

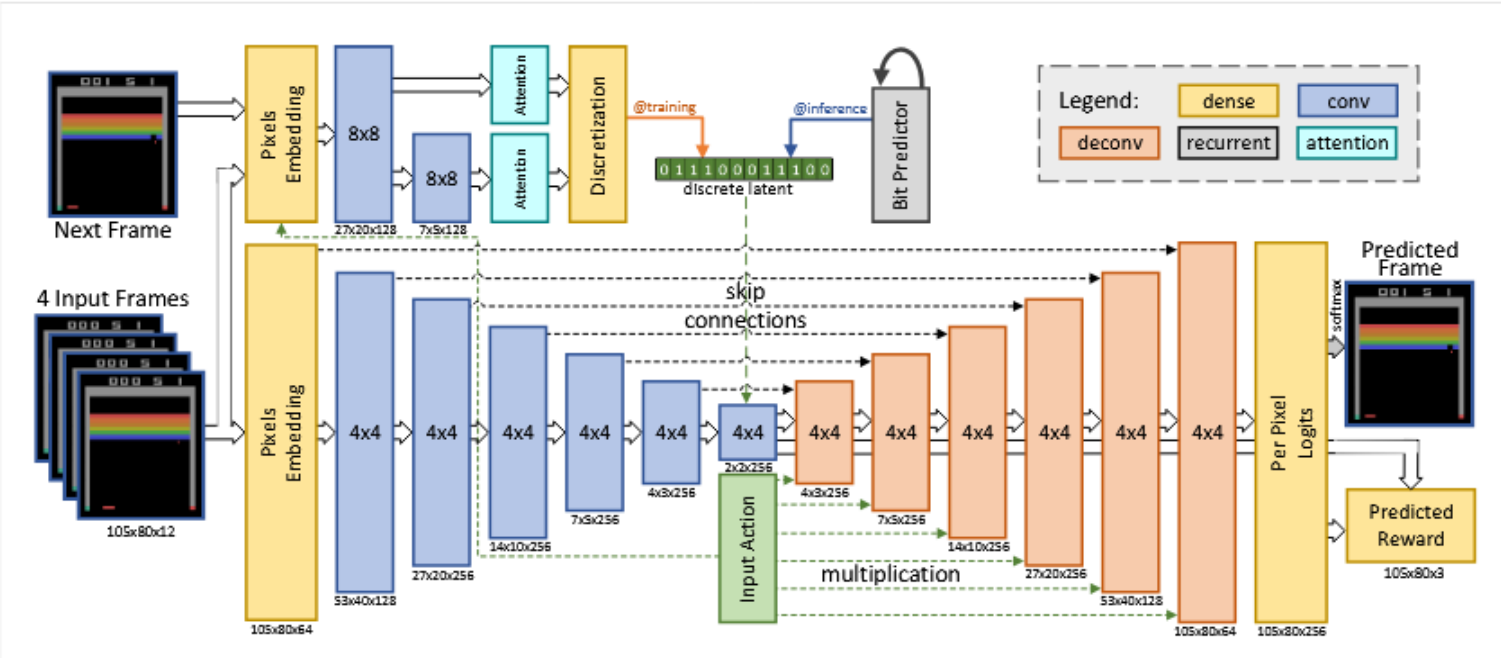


Rysunek 17: SimPLe

Kaiser, L., Babaeizadeh, M., Milos, P., Osinski, B., Campbell, R. H., Czechowski, K., ... & Michalewski, H. (2019). Model-based reinforcement learning for atari. arXiv preprint arXiv:1903.00374.

SimPLe

SimPLe



Rysunek 18: SimPLe

Kaiser, L., Babaeizadeh, M., Milos, P., Osinski, B., Campbell, R. H., Czechowski, K., ... & Michalewski, H. (2019). Model-based reinforcement learning for atari. arXiv preprint arXiv:1903.00374.

SimPLe

SimPLe

- The autoencoder compresses high-dimensional observations x_t into a latent representation z_t.
- **Encoder:** $z_t = f_\theta(x_t)$
- **Decoder:** $\hat{x}_t = g_\phi(z_t)$
- **Loss:** Reconstruction loss

$$L_{\text{recon}} = \|x_t - \hat{x}_t\|^2$$

- Typically pre-trained on a few thousand observations (e.g., 5,000–10,000 frames) before integrating into the world model.

SimPLe

SimPLe

- The world model predicts how the latent state evolves given the current state and an action.
- **Dynamics Model:** Predicts next latent state:

$$\hat{z}_{t+1} = h_\psi(z_t, a_t)$$

- **Reward Model:** Predicts reward from latent state:

$$\hat{r}_t = R_\omega(z_t, a_t)$$

- **Losses:**
  - Latent Prediction Loss:

$$L_{\text{latent}} = \|z_{t+1} - \hat{z}_{t+1}\|^2$$

  - Reward Prediction Loss:

$$L_{\text{reward}} = \|r_t - \hat{r}_t\|^2$$

- The autoencoder and world model are often fine-tuned together.

SimPLe

SimPLe

- The policy is trained in latent space using a mix of:
  - **Real interactions:** Real data is used to anchor learning.
  - **Imagined rollouts:** The world model generates simulated trajectories.
- **Policy Update:** Using an actor-critic approach:

$$\nabla_\theta L_\pi = \sum_t \nabla_\theta \log \pi(a_t \mid z_t) \cdot \left(\hat{r}_t + \gamma V(z_{t+1}) - V(z_t) \right)$$

- This combination increases sample efficiency while mitigating model bias.

SimPLe

SimPLe

- **Real Data:**
  - Collected from environment interactions.
  - Provides high-fidelity learning signals.
- **Imagined Data:**
  - Generated by simulating rollouts using the world model.
  - Offers a large number of samples at a low cost.
- Training alternates between real interactions (to update the world model) and simulated rollouts (to train the policy).

SimPLe

SimPLe

- **Autoencoder:** Learns a compact latent representation of raw observations.
- **World Model:** Predicts future latent states and rewards, enabling simulated rollouts.
- **Policy:** Trained using both real and imagined experiences in latent space.
- **Advantages:**
  - Significantly increased sample efficiency.
  - Reduced reliance on expensive real-world interactions.
  - Ability to plan and learn in a compact latent space.

Głębokie uczenie ze wzmocnieniem

I have a dream

Martin Luther King

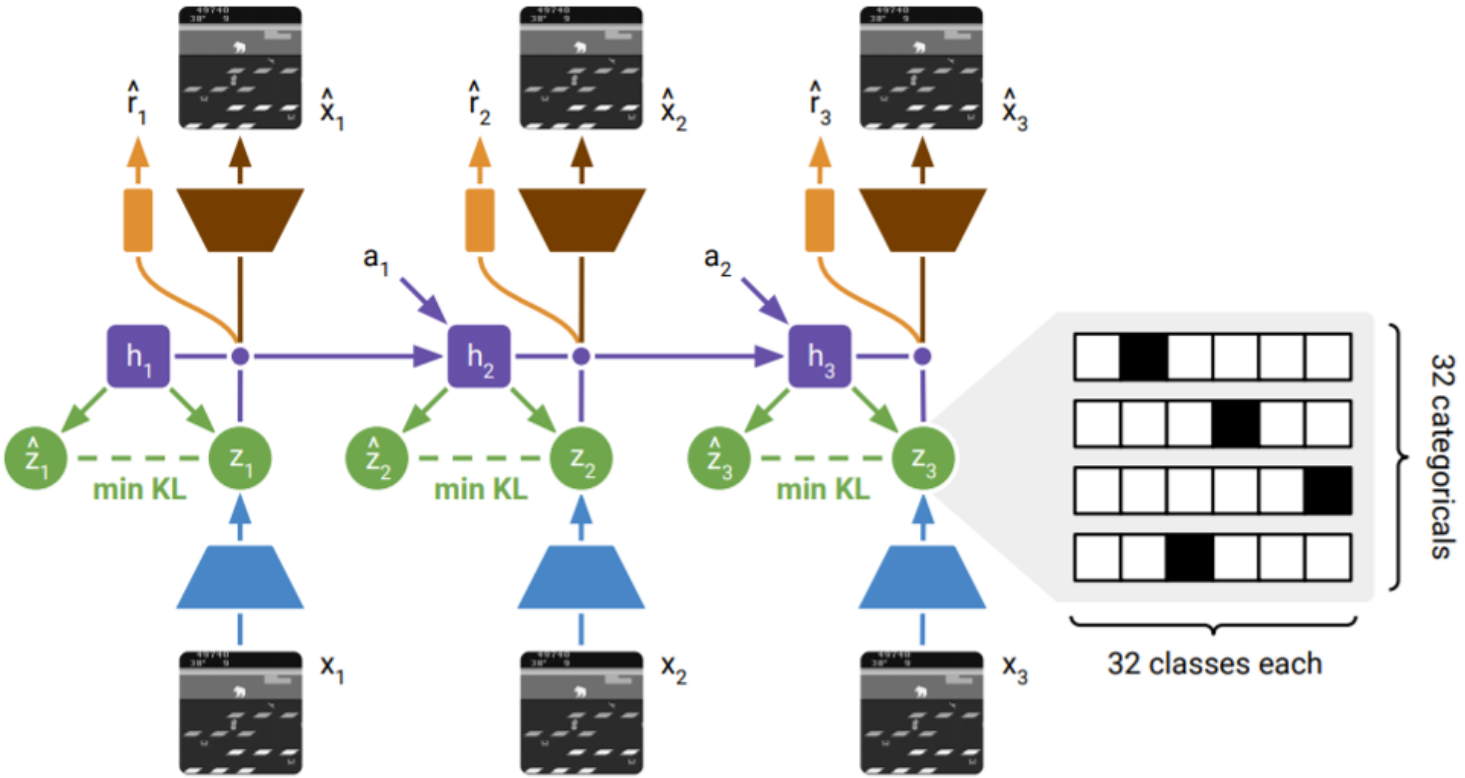
Dreamer V2

Dreamer V2

- Latent Model-Based Learning: It does not predict pixels but instead learns an abstract, compact latent space.
- Training in Imagination: After learning a model, it trains the agent inside the learned model, reducing interaction with the real world.
- Scalability: Works well with high-dimensional environments like Atari and continuous control tasks.

Dreamer V2

Dreamer V2

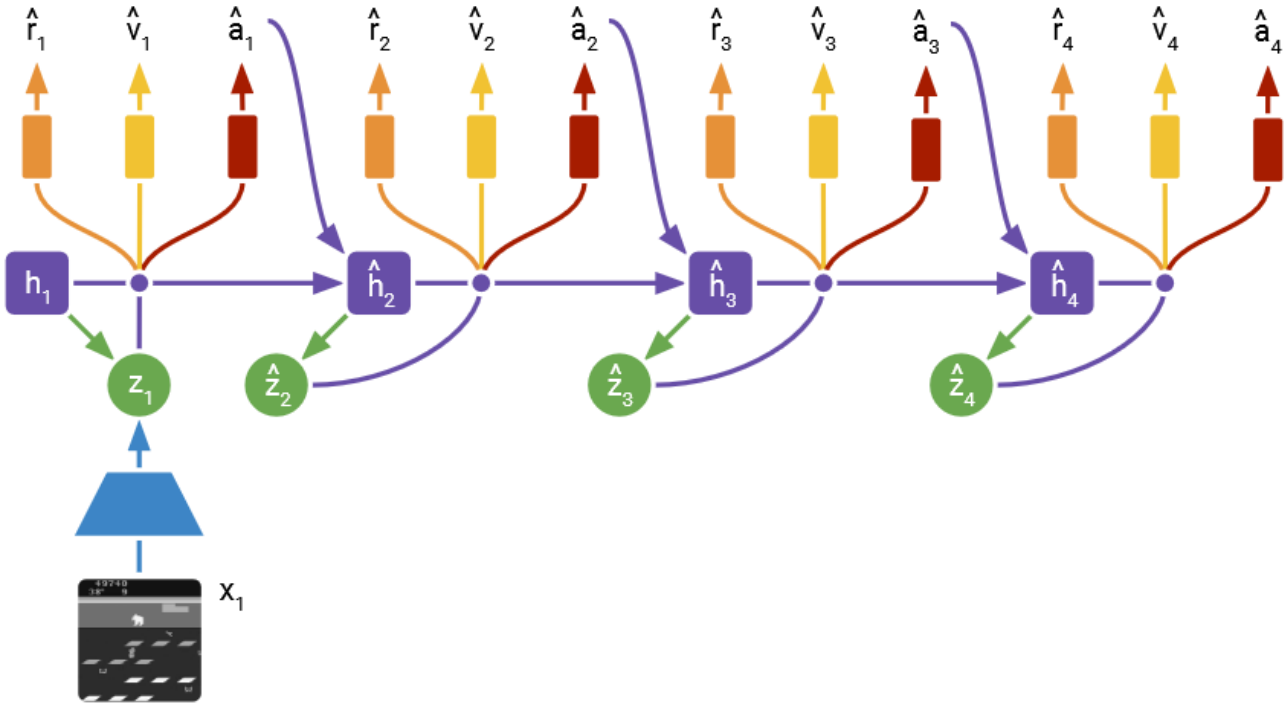


Rysunek 19: Dreamer

Hafner, D., Lillicrap, T., Norouzi, M., & Ba, J. (2020). Mastering atari with discrete world models. arXiv preprint arXiv:2010.02193.

Dreamer V2

Dreamer V2



Rysunek 20: Dreamer

Hafner, D., Lillicrap, T., Norouzi, M., & Ba, J. (2020). Mastering atari with discrete world models. arXiv preprint arXiv:2010.02193.

Dreamer V2

Dreamer V2

- Recurrent State-Space Model (RSSM): A latent dynamics model that predicts future latent states.
- Encoder $E(s)$: Converts raw observations into latent representations.
- Decoder: Converts latent states back into observations (used only for training, not planning).
- Reward and Value Networks: Predict expected rewards and values in the latent space.

Dreamer V2

Dreamer V2

$$
\begin{aligned}
\textbf{Recurrent model:} \quad & h_t = f_\phi\big(h_{t-1}, z_{t-1}, a_{t-1}\big), \\
\textbf{Representation model:} \quad & z_t \sim q_\phi\big(z_t \mid h_t, x_t\big), \\
\textbf{Transition predictor:} \quad & \hat{z}_t \sim p_\phi\big(\hat{z}_t \mid h_t\big), \\
\textbf{Image predictor:} \quad & \hat{x}_t \sim p_\phi\big(\hat{x}_t \mid h_t, z_t\big), \\
\textbf{Reward predictor:} \quad & \hat{r}_t \sim p_\phi\big(\hat{r}_t \mid h_t, z_t\big), \\
\textbf{Discount predictor:} \quad & \hat{\gamma}_t \sim p_\phi\big(\hat{\gamma}_t \mid h_t, z_t\big).
\end{aligned}
$$

Dreamer V2

Dreamer V2

- Instead of interacting with the real environment, the agent learns by imagining trajectories in the world model.
- It maximizes rewards using actor-critic learning on imagined rollouts.
- The policy updates using reinforcement learning objectives in the latent space.

Dreamer V2

Dreamer V2

- An encoder network processes each observation x_t, producing a stochastic latent variable z_t.
- The latent state z_t consists of discrete categorical variables (32 categories, 32 classes each).
- The model ensures z_t is meaningful by minimizing KL divergence between:
  - The learned posterior $q(z_t|x_t, h_t)$.
  - The prior $p(z_t|h_t)$ predicted by the model.

Dreamer V2

Dreamer V2

- The model maintains a hidden state h_t to track past information.
- It updates h_t recurrently using the previous latent state z_{t-1} and the previous action a_{t-1}:

$$h_t = f(h_{t-1}, z_{t-1}, a_{t-1})$$

- The model then predicts the prior distribution for the next latent variable z_t:

$$p(z_t|h_t)$$

Dreamer V2

Dreamer V2

$$\mathcal{L}(\phi) = \mathbb{E}_{q_\phi(z_{1:T}|a_{1:T},r_{1:T})}\Bigg[\sum_{t=1}^{T}\big(-\ln p_\theta(x_t \mid h_t, z_t) \; - \; \ln p_\theta(r_t \mid h_t, z_t)$$

$$- \; \ln p_\theta(\gamma_t \mid h_t, z_t)\big) + \; \beta \, D_{\mathrm{KL}}\big(q_\phi(z_t \mid h_t, x_t) \parallel p_\theta(z_t \mid h_t)\big)\Bigg].$$

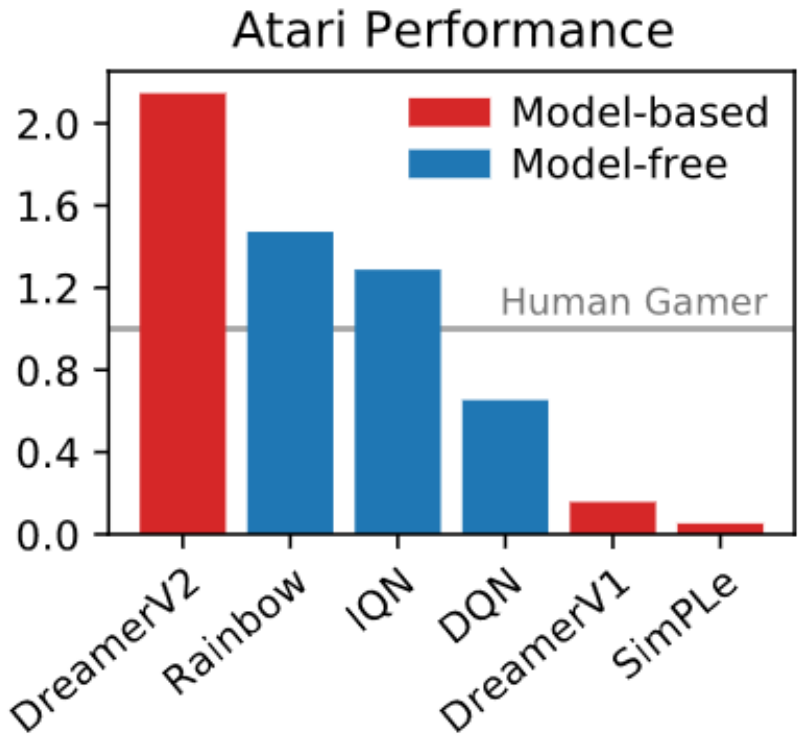
Dreamer V2

Dreamer V2

1. Collect Real Experiences
2. Encode Real Experiences into Latent Space
3. Train the World Model
4. Generate Imagined Rollouts in Latent Space
5. Train the Value Function (Critic)
6. Optimize the Policy (Actor Update)
7. Apply the New Policy in the Real Environment

Dreamer V2

Dreamer V2

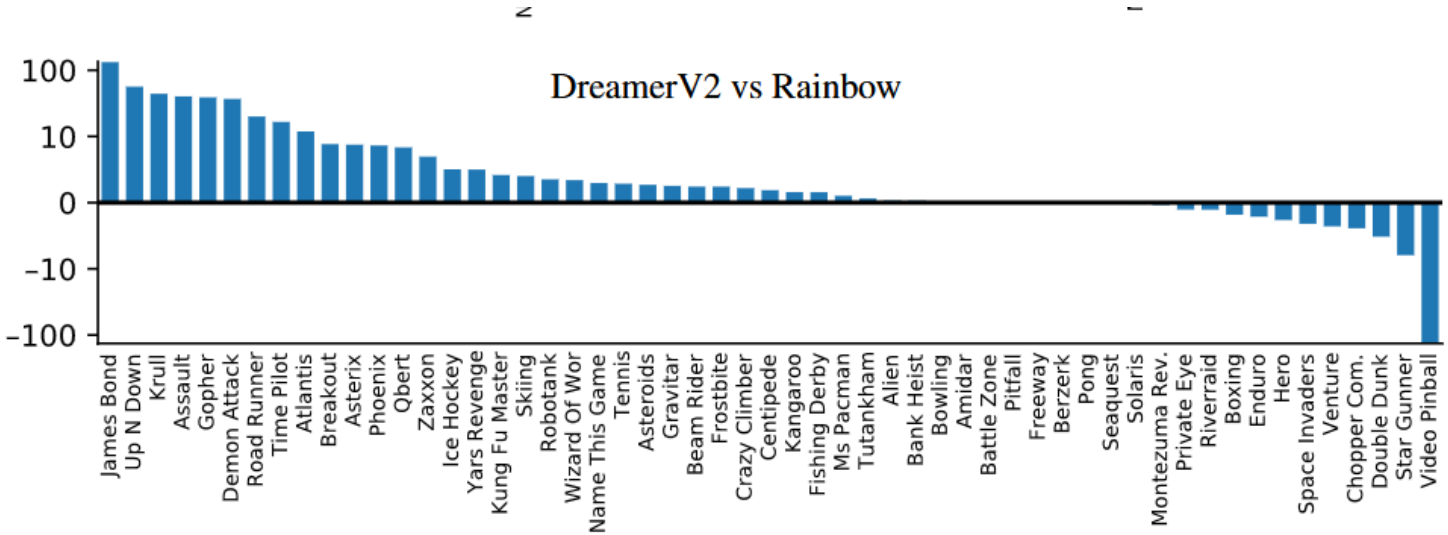


Rysunek 21: Dreamer

Hafner, D., Lillicrap, T., Norouzi, M., & Ba, J. (2020). Mastering atari with discrete world models. arXiv preprint arXiv:2010.02193.

Dreamer V2

Dreamer V2



Rysunek 22: Dreamer

Hafner, D., Lillicrap, T., Norouzi, M., & Ba, J. (2020). Mastering atari with discrete world models. arXiv preprint arXiv:2010.02193.

Głębokie uczenie ze wzmocnieniem

?

Głębokie uczenie ze wzmocnieniem

Multi-Agent Reinforcement Learning

- **Multi-Agent RL (MARL)** studies how multiple agents learn and interact in a shared environment.
- Agents may have *cooperative*, *competitive*, or *mixed* objectives.
- The presence of multiple learning agents introduces **non-stationarity** into the environment.

Głębokie uczenie ze wzmocnieniem

Multi-Agent Reinforcement Learning

- **Non-Stationarity:** Each agent's policy changes over time, affecting others.
- **Credit Assignment:** Determining individual contributions to joint outcomes.
- **Scalability:** Managing communication and coordination among many agents.
- **Partial Observability:** Agents may have limited views of the environment.

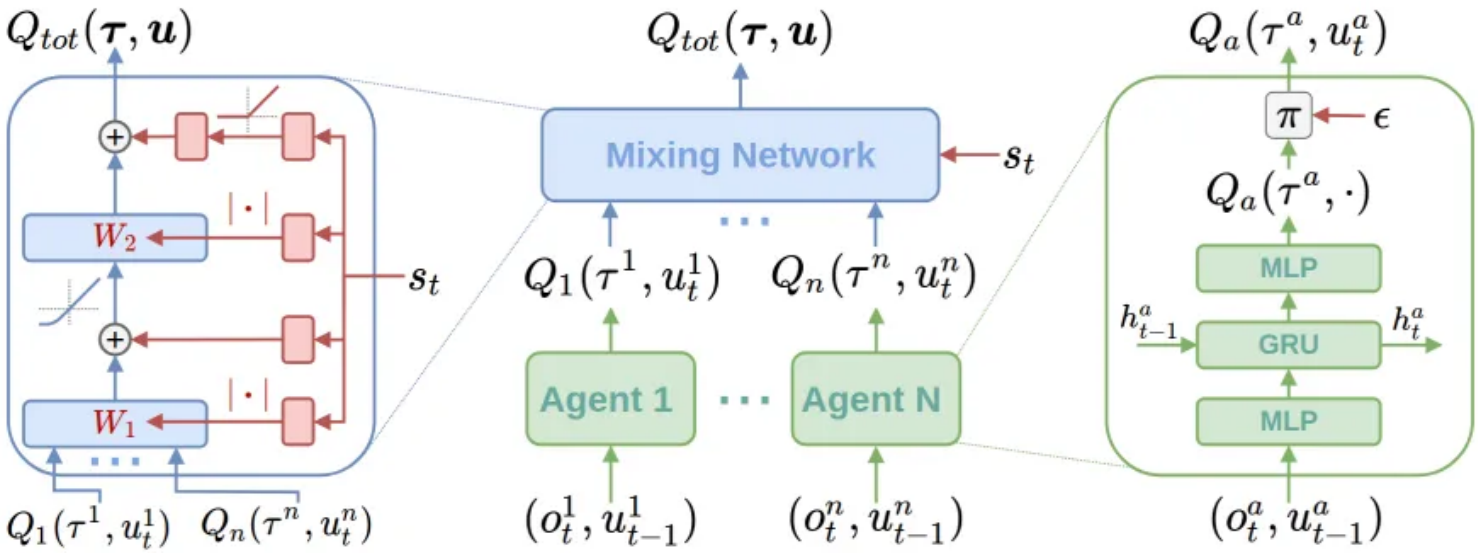
Głębokie uczenie ze wzmocnieniem

Multi-Agent Reinforcement Learning

- QMix
- Independent Proximal Policy Optimization
- Multi-Agent Proximal Policy Optimization (MAPPO)
- Multi-Agent Deep Deterministic Policy Gradient

Głębokie uczenie ze wzmocnieniem

Multi-Agent Reinforcement Learning

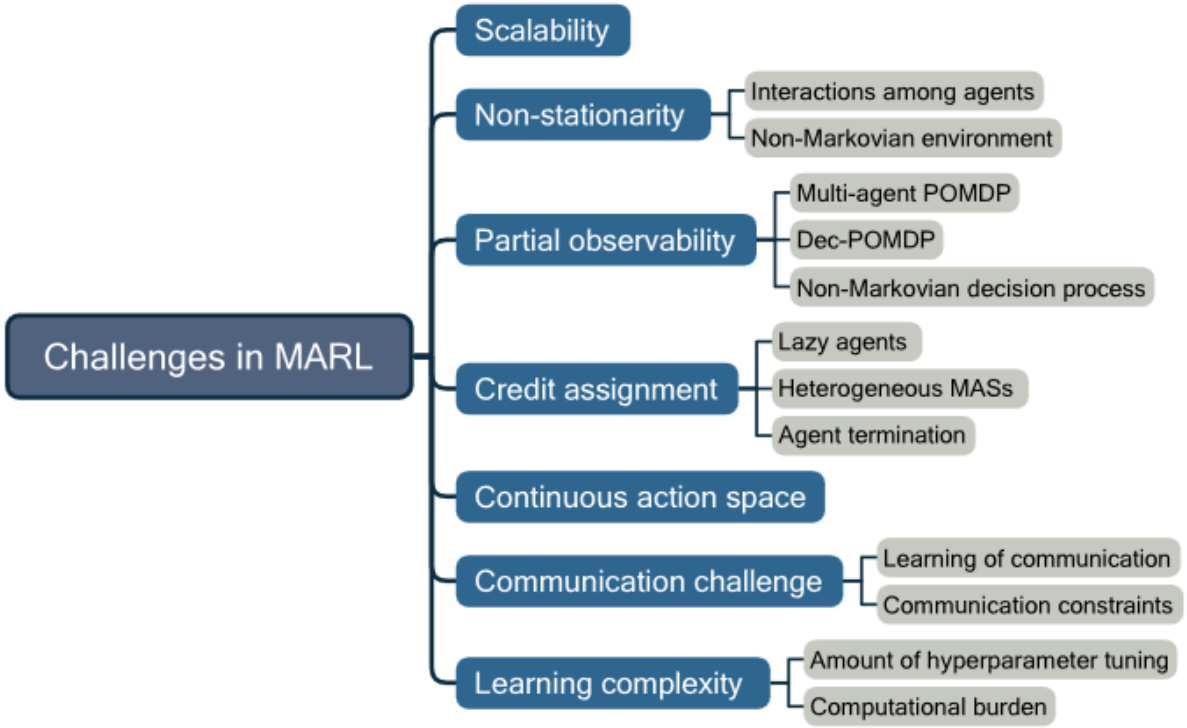


Rysunek 23: QMix

Rashid, T., Samvelyan, M., De Witt, C. S., Farquhar, G., Foerster, J., & Whiteson, S. (2020). Monotonic value function factorisation for deep multi-agent reinforcement learning. Journal of Machine Learning Research, 21(178), 1-51.

Głębokie uczenie ze wzmocnieniem

Multi-Agent Reinforcement Learning



Rysunek 24: MARL challenges

Ning, Z., & Xie, L. (2024). A survey on multi-agent reinforcement learning and its application. Journal of Automation and Intelligence, 3(2), 73-91.

Głębokie uczenie ze wzmocnieniem

Reinforcement Learning from Human Feedback (RLHF)

- **RLHF** integrates human evaluations into the reinforcement learning process to guide agent behavior.
- It addresses challenges in specifying reward functions for complex tasks by leveraging human judgment.
- This approach enables AI systems to align more closely with human preferences and values.

Głębokie uczenie ze wzmocnieniem

Reinforcement Learning from Human Feedback (RLHF)

- **Natural Language Processing:** Enhancing language models to generate more accurate and contextually appropriate responses.
- **Robotics:** Teaching robots complex tasks where explicit programming is challenging, such as household chores.
- **Content Recommendation:** Refining recommendation systems to better match user preferences through human-in-the-loop feedback.

Głębokie uczenie ze wzmocnieniem

Reinforcement Learning from AI Feedback

- **RLAIF** is an approach that utilizes AI-generated feedback to train reinforcement learning models, reducing dependence on human annotations.
- It addresses the scalability and cost challenges associated with Reinforcement Learning from Human Feedback (RLHF) by automating the feedback process.

Głębokie uczenie ze wzmocnieniem

Reinforcement Learning from AI Feedback

- **AI Preference Labeling:** An AI model assigns preference labels to generated responses, eliminating the need for human annotators.
- **Reward Model Training:** A reward model is trained using AI-generated preference labels to predict desirable outcomes.
- **Policy Optimization:** The policy model is fine-tuned through reinforcement learning, guided by the AI-trained reward model to enhance performance.

Głębokie uczenie ze wzmocnieniem

Reinforcement Learning from AI Feedback

- Developed by Anthropic, Claude leverages RLAIF to enhance its alignment with human values and ethical principles.
- Claude employs a self-supervised learning approach where it critiques and revises its responses based on predefined ethical guidelines, referred to as the "constitution."
- Through continuous self-critique and revision, Claude refines its outputs to better adhere to ethical standards without direct human intervention.

Głębokie uczenie ze wzmocnieniem

DeepSeek-R1: Using Pure RL for Thinking Tasks

- Emergent Reasoning Behaviors: The model naturally learned to verify its own reasoning, leading to more reliable outputs.
- Longer Chain-of-Thought (CoT): The model increased its test-time computation to generate more thorough responses.
- Improved Performance on Benchmarks: DeepSeek-R1 achieved results comparable to OpenAI's top models on reasoning-heavy tasks.

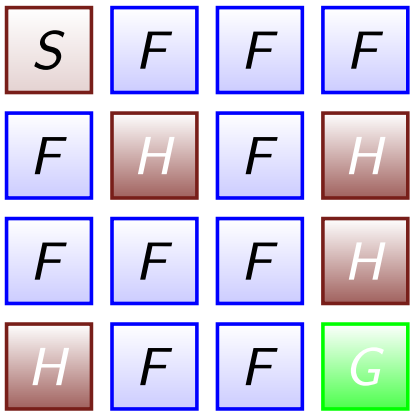
Głębokie uczenie ze wzmocnieniem

- In-Context Reinforcement Learning
- Meta Reinforcement Learning
- Hierarchical Reinforcement Learning.

Głębokie uczenie ze wzmocnieniem

Model środowiska

Frozen Lake:

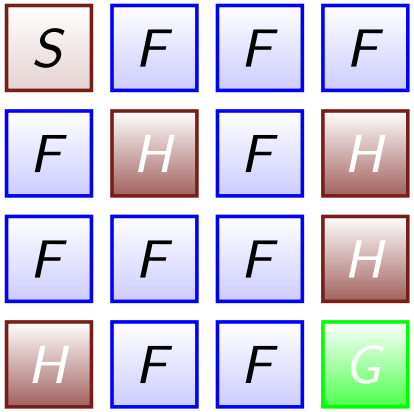


Stan jest reprezentowany za pomocą pojedynczej wartości z przedziału $< 0, 15 >$ oznaczającej pozycję agenta na planszy.
W przypadku sieci wejściem będzie wektor 16 elementowy, gdzie wartością 1 będzie zaznaczony aktualny stan - pozostałe wartości będą równe 0.

Głębokie uczenie ze wzmocnieniem

Model środowiska

Frozen Lake Extended:



Stan jest reprezentowany za pomocą trzech tablic - w pierwszej zaznaczone cyfrą 1 są dziury, w drugiej pozycja gracza, a w trzeciej cel.
W przypadku sieci wejściem będzie wektor złożony z trzech tablic.

Głębokie uczenie ze wzmocnieniem

Model środowiska

OpenAI Gym - CartPole:

Akcje:

- ruch wózka w lewo,
- ruch wózka w prawo.

Stan:

- pozycja wózka,
- prędkość wózka,
- kąt słupa,
- prędkość końcówki słupa.

Nagrody:

- 1 - za każdy krok.

Uczenie ze wzmocnieniem

Kursy online

- Reinforcement Learning:
  - *Practical Reinforcement Learning* - Yandex School of Data Analysis,
  - *Deep Reinforcement Learning* - CS 285 at UC Berkeley.
- Deep Learning:
  - *MIT 6.S191 Introduction to Deep Learning* - Massachusetts Institute of Technology,
  - *Natural Language Processing with Deep Learning* - CS224n at Stanford.

Uczenie ze wzmocnieniem

Wykłady

- *Hado Van Hasselt, Advanced Deep Learning & Reinforcement Learning*.
- *RL Course by David Silver*.
- *Deep Learning State of the Art* - MIT Deep Learning Series.

Artykuły

Artykuły

- AlphaTensor
- Zabawa w chowanego
- Dota2
- AlphaCode
- Boxing
- Pokemon Red
- InstructGPT
- EfficientZero
- Voyager
- Cicero
- Agent57
- Olimpiada matematyczna
- Stratego
- AlphaStar

VizDoom

VizDoom

- Środowisko.
- scenariusz - *DEATHMATCH*.
- mapy - **map01** (brak potworów) oraz **map03** (25 potworów).
- przydatne komendy:
  1. `game.send_game_command(f"pukename change_num_of_monster_to_spawn 0")` - zmiana liczby przeciwników,
  2. `game.send_game_command("pukename change_difficulty 5")` - zmiana poziomu trudności przeciwników (1 - 5).
- tryb *spectator* - podgląd tego co się dzieje.
- przydatne komendy:
  1. `game.get_last_action()` - pobranie ostatniej akcji,